
Remerciements

Dédicace

Abstract

This thesis presents the development of an intelligent parking management system, combining advances in computer vision and deep learning. The project is structured around two key modules : automatic license plate detection and recognition, and parking space occupancy analysis. Thanks to an optimized YOLOv11 model, remarkable performance was achieved with an mAP@50 of 87%, while the character recognition system based on PaddleOCR displays an error rate reduced to only 5%. The parking space analysis module achieves an accuracy of around 97%, illustrating the reliability of the system in real and varied conditions. This innovative solution is fully in line with the dynamics of smart cities, contributing to more efficient management of urban infrastructure and a tangible improvement in urban mobility.

Keywords : Computer vision, Deep learning, Parking management, ALPR, YOLOv11, PaddleOCR, OCR.

Résumé

Ce mémoire présente le développement d'un système intelligent de gestion de parking, combinant les avancées de la vision par ordinateur et du deep learning. Le projet s'articule autour de deux modules clés : la détection et la reconnaissance automatique des plaques d'immatriculation, ainsi que l'analyse de l'occupation des places de stationnement. Grâce à un modèle YOLOv11 optimisé, une performance remarquable a été atteinte avec un mAP@50 de 87%, tandis que le système de reconnaissance de caractères basé sur PaddleOCR affiche un taux d'erreur réduit à seulement 5%. Le module d'analyse des places de parking atteint quant à lui une précision avoisinant les 97%, illustrant la fiabilité du système dans des conditions réelles et variées. Cette solution innovante s'inscrit pleinement dans la dynamique des villes intelligentes, en contribuant à une gestion plus efficace des infrastructures urbaines et à une amélioration tangible de la mobilité urbaine.

Mots-clés : Vision par ordinateur, Deep learning, Gestion de parking, RAPI, YOLOv11, PaddleOCR, OCR.

Table des matières

Résumé

Table des matières

Table des figures

Liste des tableaux

Liste des abréviations

Introduction générale	1
1 État de l'art	3
1.1 Introduction	3
1.2 Concepts fondamentales	3
1.2.1 L'Intelligence artificielle	3
1.2.2 Machine learning	3
1.2.3 Deep learning	4
1.3 Détection d'objets et deep learning	6
1.3.1 Modèles de détection d'object	6
1.3.2 Applications de détection d'object	10
1.4 Détection et reconnaissance des plaques d'immatriculation	12
1.5 Détection de la plaque	13
1.5.1 Détection de contours (Edge detection)	13
1.5.2 Méthodes basée sur la texture (Texture based)	14
1.5.3 Convolutional neural networks CNN	14
1.5.4 You only look once YOLO	15
1.6 Reconnaissance de la plaque	15
1.6.1 Reconnaissance avec CNN	15
1.6.2 Reconnaissance avce les réseaux neuronaux récurrents	16
1.6.3 Reconnaissance optique des caractères OCR	16
1.7 Détection des places de parking	17
1.7.1 Patch-based methods	17
1.7.2 Intersection-based methods	18
1.8 Conclusion	18
2 Conception du système	19
2.1 Introduction	19
2.2 Objectifs du système	19
2.3 Architecture de système	19

2.3.1	Partie détection et reconnaissance de plaques	20
2.3.2	Partie détection des places	21
2.4	Approche proposée pour la conception du système	21
2.4.1	Détection et reconnaissance des plaque d'immatriculation	21
2.4.2	Détection des places de parking	26
2.5	Conclusion	28
3	Implémentation du système	29
3.1	Introduction	29
3.2	Environnement de développement	29
3.3	Technologies et bibliothèques utilisées	30
3.3.1	Language de programmation et outils utilisés	30
3.4	Bibliothèques utilisées	31
3.5	Vue d'ensemble du système	32
3.6	Cas d'utilisation du système	32
3.6.1	Exemple de détection et reconnaissance de matricule	32
3.6.2	Exemple de détection de places d'un parking	32
3.7	Analyse des résultats	33
3.7.1	Résultats du modèle de détection des places	34
3.7.2	Résultats du modèle de reconnaissance des plaques	36
3.7.3	Résultats du modèle de détection des places	38
3.8	Discussion	40
3.8.1	Étude des performances obtenues	40
3.8.2	Limites Identifiées	41
3.8.3	Pistes d'Amélioration	41
3.9	Conclusion	41
	Conclusion générale	42
	Bibliographie	44

Table des figures

1.1	Classification du Machine Learning	4
1.2	Architecture de R-CNN [1]	6
1.3	Architecture de SSD [2]	7
1.4	Architecture de YOLO [3]	8
1.5	Exemple de détection dans le domaine de santé [4]	10
1.6	Exemple de détection faciale [5]	10
1.7	Exemple de détection des plaques d'immatriculation [6]	11
1.8	Exemple de détection des plaques d'immatriculation	11
1.9	Exemple de plaque composée uniquement de chiffres	12
1.10	Exemple de plaque alphanumérique [7]	12
1.11	Plaque d'Immatriculation Algérienne [8]	13
2.1	Schéma d'Architecture du système	20
2.2	Exemple d'une image du dataset avec ses annotations	23
2.3	Les étapes de détection d'une plaque	23
2.4	Résultat après redimensionnement	25
2.5	Image après conversion en niveaux de gris	25
2.6	Image après réduction de bruit	25
2.7	Résultat après binarisation adaptative	26
2.8	Exemple sur les données d'entraînement du modèle de détection des places	26
2.9	Le processus de détection des places	27
3.1	Performances du modèle en termes de détection.	34
3.2	Analyse des pertes du modèle YOLOv11.	35
3.3	Évolution des métriques au fil des époques.	35
3.4	Exemple d'application sur une image.	36
3.5	Exemple d'image issue du dataset.	36
3.6	Reconnaissance de matricule : Exemple et scores WER/CER	37
3.7	Scores CER et WER moyens sur l'ensemble de test	37
3.8	Performances du modèle sur la détection des places de parking.	38
3.9	Évolution des pertes pour la détection de places.	39
3.10	Tendances des métriques par époque.	39
3.11	Exemple d'application de détection des places.	40

Liste des tableaux

2.1	Phases du système de reconnaissance de plaques	20
3.1	Résumé des performances finales du modèle	34
3.2	Performances finales du modèle de détection des places de parking	38

Liste des abréviations

AI	Artificial Intelligence (Intelligence Artificielle)
ML	Machine Learning (Apprentissage Automatique)
DL	Deep Learning (Apprentissage Profond)
IoT	Internet of Things (Internet des Objets)
CNN	Convolutional Neural Network (Réseau Neuronal Convolutif)
RNN	Recurrent Neural Network (Réseau Neuronal Récurrent)
LSTM	Long Short-Term Memory (Mémoire à Long et Court Terme)
CRNN	Convolutional Recurrent Neural Network (Réseau Neuronal Convolutif et Récurrent)
OCR	Optical Character Recognition (Reconnaissance Optique des Caractères)
ANPR	Automatic Number Plate Recognition (Reconnaissance Automatique des Plaques d'Immatriculation)
LPR	License Plate Recognition (Reconnaissance des Plaques d'Immatriculation)
YOLO	You Only Look Once (Modèle de Détection d'Objets en Temps Réel)
Faster R-CNN	Faster Region-Based Convolutional Neural Network
mAP	Mean Average Precision (Précision Moyenne Pondérée)
GPU	Graphics Processing Unit (Processeur Graphique)
API	Application Programming Interface (Interface de Programmation d'Applications)

Introduction générale

L'émergence des technologies d'intelligence artificielle (IA) et d'apprentissage profond (Deep Learning) a révolutionné divers secteurs, particulièrement dans le domaine des infrastructures urbaines intelligentes. La gestion optimisée des parkings constitue l'une des applications les plus prometteuses pour fluidifier la mobilité urbaine et atténuer les problèmes de congestion.

Notre recherche s'inscrit dans cette perspective en proposant un système innovant de gestion de parking, s'appuyant sur les dernières avancées en IA et apprentissage profond.

En milieu urbain, la gestion des parkings représente un défi croissant, exacerbé par l'augmentation constante du parc automobile et l'absence d'information en temps réel sur la disponibilité des places. Cette situation engendre non seulement des pertes de temps considérables pour les usagers, mais contribue également significativement à l'aggravation des embouteillages.

Problématique : Alors que l'approche classique de gestion des parkings en milieu urbain s'avère inefficace compte tenu de l'augmentation du nombre de véhicules, ce travail de recherche pose la question : dans quelle mesure les innovations technologiques de l'IA et du deep learning peuvent-elles s'attaquer à la fois (1) aux problématiques d'une reconnaissance fiable des plaques d'immatriculation dans des conditions réelles variables, et (2) aux enjeux d'une détection précise des places libres en temps réel ? La faisabilité d'une telle approche dépend principalement de notre aptitude à élaborer une structure technique unifiée qui puisse allier efficacité algorithmique et restrictions opérationnelles propres aux milieux urbains.

Objectifs : L'objectif principal de ce projet est de proposer une solution permettant de prédire la disponibilité des places de stationnement et de détecter et reconnaître automatiquement les plaques d'immatriculation. Ces fonctionnalités visent à améliorer l'expérience des usagers en réduisant le temps de recherche d'une place libre et en facilitant l'automatisation du contrôle d'accès aux parkings.

Plus précisément, ce travail vise à :

- Détecter automatiquement les plaques d'immatriculation avec un modèle de vision par ordinateur (YOLO).
- Extraire les caractères des plaques en texte lisible à l'aide d'un modèle OCR.
- Évaluer la performance des modèles et proposer des améliorations pour optimiser la précision et la rapidité du système.
- Prédire l'occupation des places en temps réel à l'aide de modèles de machine learning.

Ce mémoire est structuré comme suit :

- Le **Chapitre 1** présente l'état de l'art, les concepts fondamentaux et détaille les travaux existants sur la gestion intelligente des parkings et les différentes approches adoptées.
- Le **Chapitre 2** expose la méthodologie adoptée pour la conception et le développement du système proposé.
- Le **Chapitre 3** décrit l'implémentation du système, en détaillant l'environnement de travail, les bibliothèques utilisées, et les différentes étapes de développement. Il présente également l'analyse des résultats obtenus et la discussion des performances du modèle.

Chapitre 1 État de l’art

1.1 Introduction

L’essor de l’intelligence artificielle et des technologies de vision par ordinateur a profondément transformé de nombreux secteurs, notamment la gestion intelligente des infrastructures urbaines. Dans ce contexte, l’optimisation des systèmes de stationnement représente un enjeu majeur pour améliorer l’efficacité des espaces publics, réduire la congestion routière et offrir une meilleure expérience utilisateur.

Ce chapitre propose une revue de la littérature sur ces technologies. Tout d’abord, nous introduisons les fondements théoriques liés à l’intelligence artificielle et au deep learning. Ensuite, nous présentons les travaux connexes traitant des approches existantes pour la gestion intelligente des parkings. Enfin, nous discutons des défis techniques et solutions potentielles, avant de positionner notre projet dans ce contexte technologique en évolution.

1.2 Concepts fondamentales

1.2.1 L’Intelligence artificielle

L’IA est un domaine de l’informatique qui vise à développer des systèmes capables de simuler l’intelligence humaine en exécutant des tâches telles que l’apprentissage, le raisonnement et la perception. Elle se divise en plusieurs branches, notamment le machine learning, qui permet aux machines d’apprendre à partir de données, et le deep learning, qui utilise des réseaux de neurones profonds pour améliorer la reconnaissance d’images, le traitement du langage naturel et bien d’autres applications. L’IA est aujourd’hui omniprésente dans divers secteurs comme la santé, les transports et la finance, mais elle soulève également des défis liés à l’éthique, à la transparence et aux biais algorithmiques.

Et selon [9] L’IA, est un ensemble d’algorithmes conférant à une machine des capacités d’analyse et de décision lui permettant de s’adapter intelligemment aux situations en faisant des prédictions à partir de données déjà acquises.

1.2.2 Machine learning

Le ML est une méthode d’analyse de données qui automatise la création de modèles analytiques. C’est une branche de l’intelligence artificielle basée sur l’idée que les systèmes peuvent apprendre à partir des données, identifier des modèles et prendre des décisions avec une intervention humaine minimale. Ceci est maintenant largement utilisé dans tous les domaines, depuis les problèmes de classification jusqu’à conception des robot humanoïde. [10]

Les types de ML

Il existe trois types d'apprentissage :

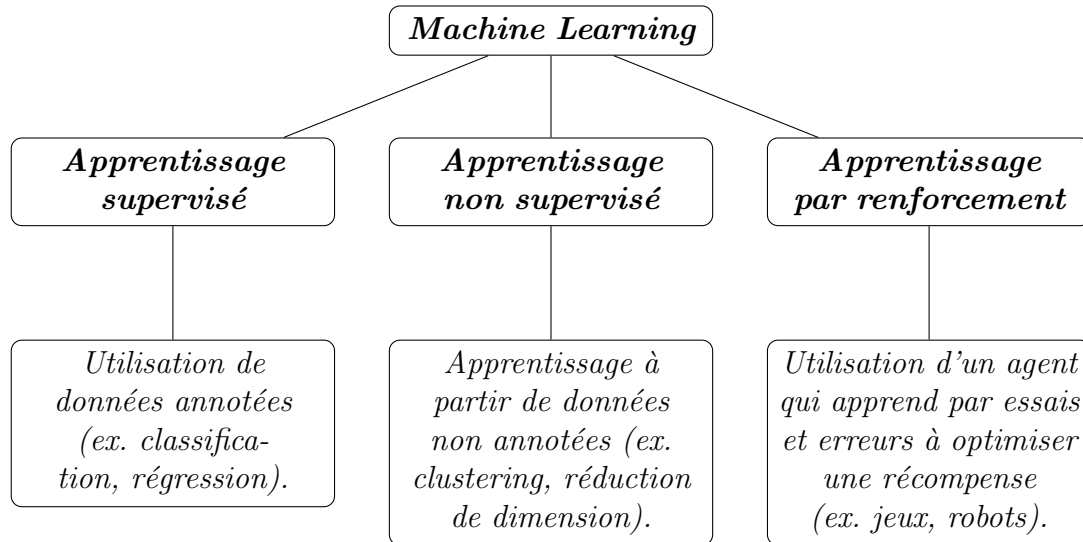


FIGURE 1.1 – Classification du Machine Learning

1.2.3 Deep learning

Le DL ou apprentissage profond est l'une des technologies principales du Machine learning. Avec le Deep Learning, nous parlons d'algorithmes capables de mimer les actions du cerveau humain grâce à des réseaux de neurones artificielles. Les réseaux sont composés de dizaines voire de centaines de «couches» de neurones, chacune recevant et interprétant les informations de la couche précédente. [11]

Ces modèles sont capables d'extraire automatiquement des représentations complexes à partir des données brutes, ce qui les rend particulièrement efficaces pour des tâches comme la vision par ordinateur et le traitement du langage naturel.

Les réseaux de neurones artificiels

Un réseau de neurones artificiel (ANN) est un modèle inspiré du fonctionnement du cerveau humain. Il est composé de neurones artificiels organisés en trois types de couches :

- La couche d'entrée qui reçoit les données brutes.
- Les couches cachées où se font les calculs et l'apprentissage des caractéristiques.
- La couche de sortie qui génère la prédiction finale.

Les différents algorithmes de Deep Learning

Plusieurs types d'algorithmes sont utilisés en apprentissage profond. Chaque algorithme possède ses propres spécificités et applications, et parmi ces algorithmes nous citons les plus utilisés :

- **Réseaux neuronaux convolutifs (CNN)** : Spécialisés dans l'analyse d'images, les CNN utilisent des couches de convolution pour extraire des caractéristiques. Ils sont largement employés en vision par ordinateur, notamment pour la reconnaissance et la détection d'objets. [12]

- **Réseaux neuronaux récurrents (RNN)** : Adaptés aux données séquentielles, les RNN possèdent des connexions récurrentes permettant de mémoriser les entrées précédentes. Ils sont utilisés dans le traitement du langage naturel et la traduction automatique. [12]
- **Réseaux de neurones profonds (DNN)** : Composés de plusieurs couches cachées, les DNN apprennent des représentations complexes des données. Ils sont utilisés en classification, reconnaissance d'images et prédiction, et servent de base aux CNN et RNN.
- **Réseaux de mémoire à long et court terme (LSTM)** : Variante des RNN, les LSTM capturent les dépendances à long terme grâce à des cellules de mémoire. Ils sont utilisés pour la prédiction de séries temporelles, la reconnaissance vocale et la musique générative. [12]

Étapes de deep learning

L'apprentissage profond suit plusieurs étapes essentielles pour entraîner un modèle efficace. Ces étapes garantissent une bonne généralisation et une performance optimale du modèle sur de nouvelles données. Voici les principales étapes :

1. **Collecte et préparation des données** : Acquisition, nettoyage et annotation des données.
2. **Prétraitement des données** : Augmentation des données et transformation des images.
3. **Choix de l'architecture du modèle** : Sélection du type de réseau neuronal (CNN, RNN, etc.).
4. **Entraînement du modèle** : Ajustement des poids du réseau à l'aide d'un algorithme d'optimisation (SGD, Adam, etc.).
5. **Évaluation et validation** : Mesure des performances à l'aide de métriques comme la précision et le rappel.
6. **Optimisation et réglage des hyperparamètres** : Ajustement des paramètres pour améliorer les performances.
7. **Déploiement et inférence** : Utilisation du modèle en production pour faire des prédictions sur de nouvelles données.

1.3 Détection d'objets et deep learning

La détection d'objets est un domaine important du traitement d'images et de la vision par ordinateur. Elle repose sur l'utilisation de techniques d'apprentissage profond et de vision par ordinateur pour identifier divers types d'objets dans des flux vidéo et des fichiers. Alors que les humains reconnaissent naturellement les objets du monde réel, les machines nécessitent des algorithmes avancés pour accomplir cette tâche. La détection d'objets est un aspect clé de la vision par ordinateur, consistant à localiser et identifier des objets spécifiques dans une image. La reconnaissance d'objets, quant à elle, est une tâche fondamentale de la vision par ordinateur qui permet de détecter et d'identifier des objets dans des images numériques, ainsi que dans des vidéos stockées ou en temps réel. En d'autres termes, la reconnaissance d'objets consiste à déterminer l'identité d'un objet observé dans une image ou une vidéo. [13]

1.3.1 Modèles de détection d'objet

La détection d'objets repose principalement sur deux approches : les modèles à étape unique (single-stage) et les modèles à deux étapes (two-stage). Dans les sections suivantes, nous détaillerons quelques exemples emblématiques de chacune de ces catégories.

R-CNN (Regions with Convolutional Neural Networks)

La conception de RCNN est simple : elle commence par extraire un ensemble de propositions d'objets (boîtes candidates) à l'aide de la recherche sélective. Ensuite, chaque proposition est redimensionnée à une image de taille fixe et transmise à un modèle CNN entraîné sur ImageNet (par exemple, AlexNet) pour en extraire les caractéristiques. Enfin, des classificateurs SVM linéaires sont utilisés pour prédire la présence d'un objet dans chaque région et reconnaître les catégories d'objets. [14]

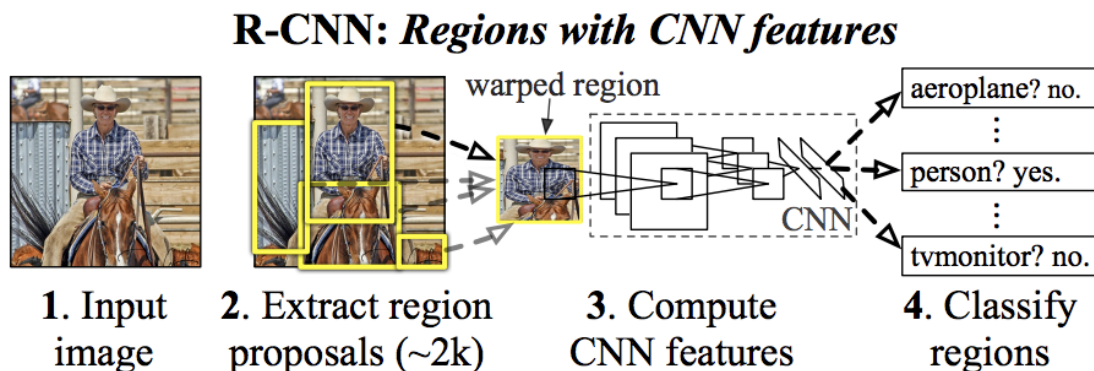


FIGURE 1.2 – Architecture de R-CNN [1]

SSD (Single Shot MultiBox Detector)

SSD a été proposé par W. Liu et al. en 2015. Il s'agit du deuxième détecteur en une seule étape de l'ère du deep learning. La principale contribution de SSD est l'introduction des techniques de détection multi-référence et multi-résolution, qui améliorent significativement la précision de détection des détecteurs en une seule étape, notamment pour les petits objets. Le SSD présente des avantages en termes de vitesse et de précision avec une version rapide fonctionnant à 59 images par seconde. La principale différence entre SSD et les détecteurs précédents est que SSD détecte les objets à différentes échelles sur différentes couches du réseau, alors que les autres détecteurs ne réalisent la détection que sur leurs couches supérieures.

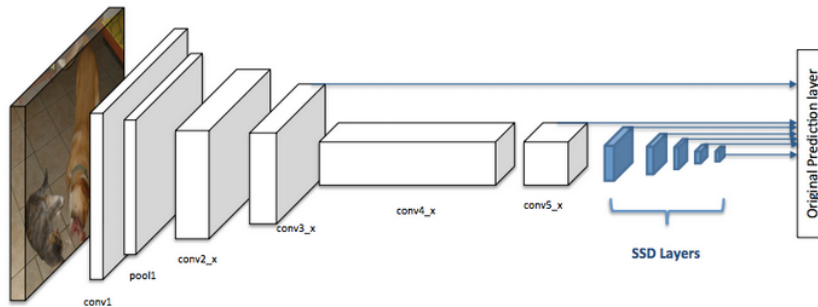


FIGURE 1.3 – Architecture de SSD [2]

YOLO (You Only Look Once)

YOLO est un algorithme de détection d'objets populaire qui peut détecter des objets en temps réel en divisant l'image en une grille de cellules et en prédisant la classe et l'emplacement de l'objet dans chaque cellule. YOLO peut détecter plusieurs objets dans une image [15]. YOLO a été proposé par R. Joseph et al. en 2015. Il s'agit du premier détecteur en une seule étape de l'ère du deep learning. il est extrêmement rapide : une version optimisée atteint 155 images par seconde (fps) avec une mAP de 52,7 % sur VOC07, tandis qu'une version améliorée fonctionne à 45 fps avec 63,4 % de mAP sur VOC07 et 57,9 % sur VOC12.

YOLO comme son nom l'indique, ses auteurs ont totalement abandonné l'approche classique de la détection par (proposition + vérification). À la place, il applique un unique réseau de neurones à l'image complète. Ce réseau divise l'image en plusieurs régions et prédit simultanément les boîtes englobantes et les probabilités associées. Par la suite, R. Joseph a apporté plusieurs améliorations et a proposé des versions qui augmentent encore la précision de détection tout en conservant une vitesse très élevée. [14]

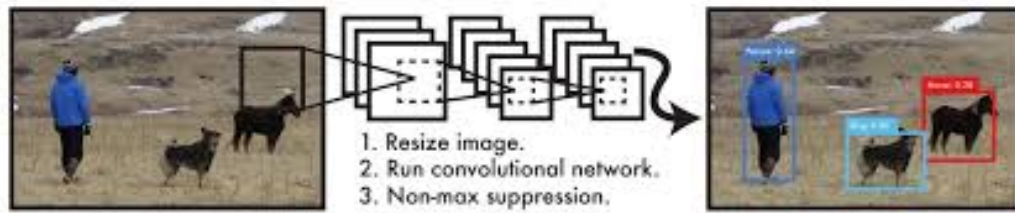


FIGURE 1.4 – Architecture de YOLO [3]

L'évolution récente des versions de YOLO a conduit au développement de YOLOv11, qui apporte des améliorations significatives en termes de précision et de rapidité

1.3.1.1 C'est quoi le YOLOv11 ?

YOLOv11 est la dernière itération de la série YOLO, s'appuyant sur les bases établies par YOLOv1. Présenté lors de la conférence YOLO Vision 2024 (YV24), YOLOv11 représente une avancée majeure dans la technologie de détection d'objets en temps réel. Cette nouvelle version apporte des améliorations significatives tant au niveau de l'architecture que des méthodologies d'entraînement, repoussant ainsi les limites en termes de précision, de rapidité et d'efficacité.[16]

1.3.1.2 Architecture de YOLOv11

YOLOv11 est une avancée notable dans les modèles de détection d'objets en temps réel. Son architecture se distingue par plusieurs améliorations clés qui renforcent ses capacités de détection tout en optimisant l'efficacité computationnelle. L'architecture de YOLO repose sur trois composants fondamentaux :

1. **Backbone** : Le *backbone* est responsable de l'extraction des caractéristiques à partir de l'image d'entrée. YOLOv11 introduit plusieurs améliorations dans cette phase :
 - **C3k2 block** : Une version plus rapide et efficace du CSP Bottleneck, utilisant deux petites convolutions au lieu d'une grande, ce qui réduit la complexité tout en maintenant les performances.
 - **C2PSA block** : Un mécanisme d'attention spatiale permettant de se concentrer sur les régions importantes de l'image, améliorant ainsi la détection des objets à différentes échelles.
2. **Neck** : Le *neck* combine les caractéristiques extraites à différentes échelles pour les transmettre au *head* :
 - **Agrégation multi-échelle** : Utilisation de l'*upsampling* et de la *concaténation* pour enrichir les informations issues du *backbone*.
 - **C3k2 block** : Remplacement du bloc C2f par le C3k2, offrant une meilleure agrégation des caractéristiques et réduisant la complexité du modèle.
3. **Head** : Le *head* génère les prédictions finales à partir des cartes de caractéristiques traitées par le *neck* :
 - **C3k2 blocks** : Utilisation de plusieurs blocs C3k2 pour raffiner les cartes de caractéristiques et améliorer la précision des prédictions.

- **CBS blocks** : Intégration de couches *Convolution-BatchNorm-SiLU* (CBS) pour stabiliser le flux de données et affiner les prédictions.
- **Détection finale** : Les couches de convolution finales et la *Detect layer* produisent les coordonnées des boîtes englobantes, les scores d'objets et les classes détectées. [17]

1.3.1.3 Comment YOLOv11 détecte-t-il un objet ?

YOLOv11 détecte un objet en plusieurs étapes :

1. **Annotation des données** : Avant l'entraînement, les images sont annotées avec des boîtes englobantes (*bounding boxes*) qui délimitent l'objet que l'on veut détecter. Chaque annotation contient les coordonnées de la boîte et la classe associée.
2. **Division de l'image en grille** : L'image est divisée en une grille de cellules. Chaque cellule est responsable de prédire s'il y a un objet dans sa zone et propose plusieurs boîtes englobantes de tailles différentes pour s'adapter à la forme de l'objet.
3. **Prédiction des boîtes** : Pour chaque cellule, YOLOv11 prédit :
 - Les coordonnées des boîtes englobantes.
 - Un score de confiance, qui indique la probabilité qu'un objet soit présent.
 - La classe de l'objet détecté.
4. **Filtrage** : Pour éviter les doublons, une étape appelée *Non-Maximum Suppression* (NMS) est appliquée. Cette étape garde uniquement les boîtes avec les scores les plus élevés et élimine celles qui se chevauchent trop.
5. **Résultat final** : Après le filtrage, les boîtes finales indiquent précisément l'emplacement de l'objet dans l'image.

1.3.2 Applications de détection d'objet

Afin de mieux comprendre les avancées en détection d'objets, nous proposons dans cette section une classification des modèles existants, accompagnée d'exemples représentatifs.

1.3.2.1 Santé et imagerie médicale

Dans l'imagerie médicale, la détection d'objets est un outil essentiel pour aider les professionnels de santé à identifier et diagnostiquer diverses affections. Cette technique permet de détecter et segmenter différentes structures au sein des images médicales, telles que les tumeurs, organes ou vaisseaux sanguins. Les résultats fournis par les algorithmes de détection d'objets apportent des informations précieuses, facilitant ainsi la prise de décisions médicales. En automatisant l'identification de ces structures, la détection d'objets en imagerie médicale gagne du temps, réduit les erreurs humaines et améliore la précision des diagnostics.[13]

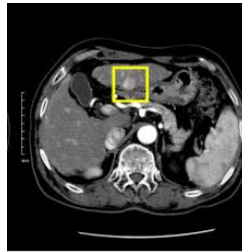


FIGURE 1.5 – Exemple de détection dans le domaine de santé [4]

1.3.2.2 Reconnaissance faciale

Les visages humains fournissent une richesse d'informations permettant de mieux comprendre l'état d'esprit d'un utilisateur, ce qui en fait un outil précieux pour améliorer les interactions homme-machine. Au cours de la dernière décennie, la classification et la reconnaissance des expressions faciales ont fait l'objet de nombreuses études, aboutissant à diverses applications. Parmi elles, on trouve les robots socialement intelligents dotés d'intelligence émotionnelle, les systèmes d'assistance pour les personnes autistes, les détecteurs d'émotions pour les personnes en situation de handicap, les systèmes de détection de douleur ou de stress pour les études psychologiques, les tuteurs intelligents et les applications mobiles capables d'insérer automatiquement des émotions dans les discussions en ligne. [18]

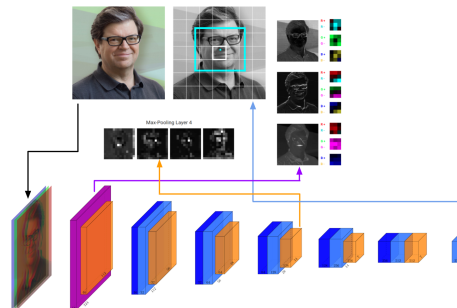


FIGURE 1.6 – Exemple de détection faciale [5]

1.3.2.3 Plaque d'immatriculation

Les systèmes de Reconnaissance Automatique des Plaques d'Immatriculation (ALPR) gagnent en popularité en raison de leur large éventail d'applications dans les systèmes de transport intelligents. Ils sont utilisés pour l'application des lois de la circulation, la surveillance du trafic, la gestion des parkings, la collecte des péages et le contrôle de sécurité dans des zones restreintes comme les camps militaires et les sites protégés. Ces systèmes permettent de prévenir la fraude et renforcer la sécurité en facilitant la recherche de véhicules volés ou impliqués dans des crimes. Sans les systèmes ALPR, ces tâches nécessiteraient d'importants efforts humains, du temps et des ressources. De plus, une intervention manuelle pourrait entraîner des erreurs d'interprétation, et il est pratiquement difficile pour un humain de mémoriser ou lire efficacement la plaque d'un véhicule en mouvement. [19]

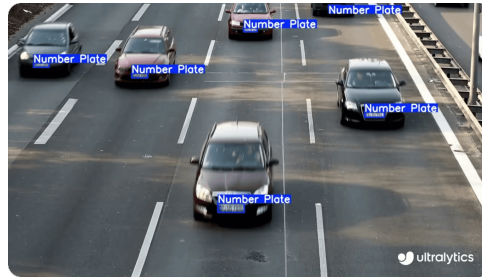


FIGURE 1.7 – Exemple de détection des plaques d'immatriculation [6]

1.3.2.4 Détection de l'occupation des places de parking

La détection de l'occupation des places de parking est un domaine d'application important de la détection d'objets, visant à optimiser la gestion des espaces de stationnement. Martynova et al. [20] ont mené une étude approfondie sur les méthodes de détection de l'occupation des parkings en conditions réelles. Ils ont identifié que les approches existantes manquent souvent de robustesse face à des conditions visuelles variées, telles que les changements d'éclairage ou les conditions météorologiques. Pour pallier ces limites, les auteurs ont proposé une méthode reposant sur l'architecture EfficientNet, qui a montré de meilleures performances sur plusieurs ensembles de données. De plus, ils ont introduit le dataset SPKL (Seasonal Parking Lot), composé d'images capturant différentes situations réelles, comme des parkings enneigés ou partiellement obstrués, offrant ainsi une évaluation plus représentative des systèmes de détection.

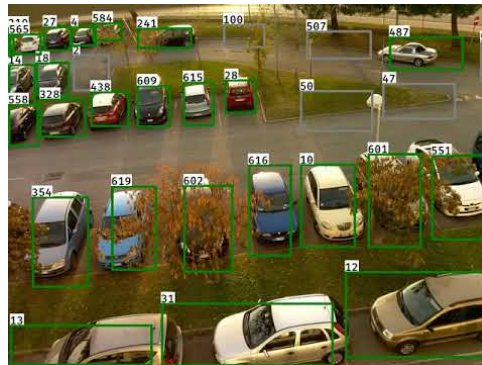


FIGURE 1.8 – Exemple de détection des plaques d'immatriculation

1.4 Détection et reconnaissance des plaques d'immatriculation

ALPR signifie "Automatic License Plate Recognition" (Reconnaissance Automatique des Plaques d'Immatriculation) et désigne le processus d'identification et d'extraction de la zone de la plaque d'immatriculation d'un véhicule à partir d'une image, sans intervention humaine. Il s'agit d'une étape cruciale dans un système de reconnaissance automatique des plaques d'immatriculation, qui comprend trois phases principales : l'acquisition d'image, la détection de la zone de la plaque et l'extraction ainsi que la reconnaissance des caractères. La phase de détection est particulièrement importante, car elle doit gérer plusieurs défis, notamment la variation de l'emplacement des plaques, la présence de plusieurs plaques dans une même image, les différences de tailles et de couleurs des plaques, l'inclinaison de la caméra et les mauvaises conditions d'éclairage. En raison de ces difficultés, l'ALPR est considéré comme un sujet de recherche complexe dans le domaine du traitement d'images.

Différentes formes de plaques d'Immatriculation

Les plaques d'immatriculation varient non seulement en fonction du pays, mais aussi selon la structure des caractères qu'elles affichent. On peut distinguer plusieurs formes courantes :

1. Plaques composées uniquement de chiffres

- Présentes dans plusieurs pays, comme l'Algérie et certains pays du Moyen-Orient.
- Exemple :

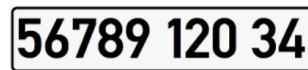


FIGURE 1.9 – Exemple de plaque composée uniquement de chiffres

2. Plaques alphanumériques (chiffres + lettres)

- Utilisées en Europe et en Amérique du Nord, combinant lettres et chiffres pour plus de possibilités d'immatriculation.
- Exemple :



FIGURE 1.10 – Exemple de plaque alphanumérique [7]

Caractéristiques de plaque algériennne

La plaque d'immatriculation en Algérie permet, comme tous les types de plaques minéralogiques, d'identifier les véhicules. Elles suivent un format spécifique et elles sont composées de trois groupes de chiffres, séparés par des espaces :

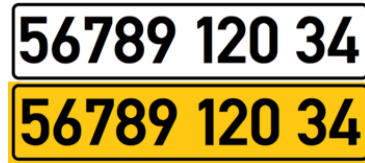


FIGURE 1.11 – Plaque d'Immatriculation Algérienne [8]

- **1^{er} groupe** : Ensemble de chiffres , correspondant au numéro de dossier du véhicule.
- **2^e groupe** : Trois chiffres :
 - Le premier chiffre identifie le type de véhicule :
 - 1 : Voiture
 - 2 : Camion
 - 3 : Camionnette
 - 4 : Véhicule de transport ou autobus
 - 5 : Tracteur routier
 - 6 : Autre type de tracteur
 - 7 : Véhicule spécial
 - 8 : Remorque
 - 9 : Moto
 - Les deux chiffres suivants indiquent l'année de mise en circulation du véhicule.
- **3^e groupe** : Deux chiffres identifiant la wilaya d'immatriculation de 01 à 58.

1.5 Détection de la plaque

Dans cette section, nous nous intéressons à la détection de plaques d'immatriculation. Nous présenterons différentes approches existantes, en mettant en évidence leurs avantages et leurs limites.

1.5.1 Détection de contours (Edge detection)

L'algorithme proposé dans [21] pour la détection des plaques d'immatriculation des véhicules est basé sur le nombre d'Euler d'une image binaire et l'opérateur Mexican Hat pour la détection des contours. Bien que les auteurs revendiquent un taux de réussite élevé de 94 à 99 % et une précision moyenne de 96,17 %, leur algorithme présente une limite majeure. Si la plaque d'immatriculation est noire, le système de détection des contours peut ne pas réussir à identifier correctement les bords de la plaque. Cette défaillance est due au fait que l'opérateur Mexican Hat, utilisé pour la détection des contours, est un filtre passe-haut qui amplifie les composantes haute fréquence de l'image. Dans le cas d'une plaque noire, les contours peuvent ne pas contenir suffisamment de ces composantes, ce qui empêche leur détection par l'algorithme.

le système de détection des plaques d'immatriculation proposé par Chen constitue une solution prometteuse pour la détection des plaques dans des environnements variés et sous différentes conditions d'éclairage. Grâce à l'utilisation d'un opérateur Prewitt amélioré et d'une méthode de projection, ce système offre une haute précision et une efficacité en temps réel, le rendant ainsi adapté à des applications pratiques, notamment pour la surveillance du trafic et l'application des lois [22].

1.5.2 Méthodes basée sur la texture (Texture based)

Dans [23], une méthode de détection des plaques d'immatriculation basée sur la transformation en ondelettes est décrite. Cette approche exploite la sous-bande HL de la transformation en ondelettes pour extraire des caractéristiques et utilise ensuite la sous-bande LH pour vérifier la présence d'une ligne horizontale autour de la caractéristique détectée. Selon les auteurs, cette méthode atteint une précision de 97,33 % pour la localisation des plaques. L'un des avantages des méthodes basées sur la texture, comme celle décrite dans [23], est leur robustesse face à la déformation des plaques d'immatriculation. En effet, cette approche permet de détecter et de reconnaître correctement les plaques même lorsqu'elles sont pliées ou déformées, ce qui est essentiel étant donné que les plaques peuvent être altérées par des dommages ou l'usure naturelle.

Dans [24], une technique basée sur les lignes de balayage (scan-line) a été utilisée pour la détection des plaques d'immatriculation. Cette approche repose sur l'observation que la complexité de la région contenant la plaque dans une image en niveaux de gris est unique et diffère du reste de l'image. Ainsi, cette méthode ne dépend pas des contours de la plaque, ce qui la rend robuste dans divers scénarios.

Zunino et al. [25] ont proposé une nouvelle approche de localisation basée sur la quantification vectorielle (VQ). Contrairement aux méthodes qui s'appuient uniquement sur les bords et le contraste, cette approche prend en compte le contenu réel de la plaque d'immatriculation. Les auteurs rapportent une précision de détection de 98 %, testée dans une application industrielle en temps réel.

1.5.3 Convolutional neural networks CNN

Les auteurs de [26] ont proposé une approche basée sur un CNN pour la détection des plaques d'immatriculation, capable de gérer des images floues et diverses conditions d'éclairage. Leur modèle, entraîné sur un grand jeu de données, a atteint une précision de 100 % sur leur jeu de test. Toutefois, cette performance pourrait ne pas être garantie en conditions réelles en raison de divers facteurs environnementaux.

D'autres études, comme celle de Selmi et al. [27], ont adopté une approche similaire en combinant prétraitement et classification via un CNN, atteignant une rappel de 93,8 % et un score F1 de 91,3 % sur le jeu de données Caltech.

Zou et al. [28] ont proposé une architecture à deux CNNs : un shallow CNN pour réduire le coût computationnel et un deep CNN pour localiser précisément la plaque, suivi d'une suppression des non-maxima (NMS). Leur méthode a montré une précision élevée avec un coût réduit.

1.5.4 You only look once YOLO

Silva et al. [29] ont utilisé YOLOv2 pour détecter les véhicules sans ajustement spécifique. En parallèle, WPOD-NET, basé sur YOLO, SSD et STN, a été conçu pour rectifier les plaques déformées en les transformant en vues frontales.

Min [30] a proposé un système de localisation des plaques utilisant YOLO-L et une méthode de pré-identification des plaques, combinée à un algorithme de k-means pour optimiser les boîtes de détection. Leur approche a atteint une précision et un rappel de 98,86 % sur un jeu de données varié.

Xie et al. [31] ont amélioré YOLO avec MDYOLO, capable de gérer les plaques multi-orientées en prédictant leur angle de rotation. En modifiant l'architecture CNN et en introduisant une nouvelle fonction de perte, ils ont obtenu une meilleure robustesse aux conditions d'éclairage difficiles et aux occlusions.

1.6 Reconnaissance de la plaque

Après la détection de la plaque, la reconnaissance constitue une étape essentielle. Dans cette section, nous explorerons différentes approches de reconnaissance de caractères appliquées aux plaques d'immatriculation.

1.6.1 Reconnaissance avec CNN

Alam [32] a proposé une méthode de reconnaissance des plaques d'immatriculation en bengali basée sur un réseau de neurones convolutionnel (CNN). Son approche utilise une technique de super-résolution pour améliorer la qualité des images, permettant une meilleure segmentation et identification des caractères. Les tests sur 700 véhicules ont montré une précision de 98.2% en validation et 98.1% en évaluation.

Rahman [33] a développé un système de reconnaissance des plaques Bangla (BLPRS) basé sur un CNN à six couches convolutives et une couche entièrement connectée. Ce système, conçu pour l'assistance routière et l'identification des véhicules, a atteint une précision de 89% sur l'ensemble de test.

Liu et al. [34] ont proposé une approche combinant analyse des composants connectés et projection pour la segmentation des plaques, ainsi que deux CNNs pour la reconnaissance des caractères (SCNN et RCNN). Leur méthode traite les plaques surexposées via une conversion en niveaux de gris et une amélioration du contraste. Avec 2189 images d'entraînement, leur système a atteint un taux de segmentation de 96.58% et un taux de reconnaissance de 98.09%, surpassant les approches existantes en termes de précision et de robustesse.

1.6.2 Reconnaissance avec les réseaux neuronaux récurrents

Li et al. [35] ont proposé un système de reconnaissance des plaques d'immatriculation basé sur un Réseau de Neurones Récurrents Bidirectionnel (BRNN) avec une perte CTC (Connectionist Temporal Classification). L'architecture utilise un RPN (Region Proposal Network) pour extraire les caractéristiques des plaques, puis un BRNN pour capturer le contexte à la fois dans le passé et le futur. La perte CTC permet d'apprendre les séquences de caractères sans alignement explicite, ce qui améliore la précision globale du modèle sur divers jeux de données.

L'étude de [36] propose une approche combinant RNNs avec LSTM et CNNs pour extraire les caractéristiques séquentielles des plaques d'immatriculation. Chaque image détectée est convertie en niveaux de gris (24x94 pixels), puis segmentée via une technique de fenêtre glissante (24x24 pixels). Un CNN de 36 classes extrait les caractéristiques, réduites ensuite à 256 dimensions avec PCA. La perte CTC est utilisée pour décoder les prédictions, atteignant un taux de reconnaissance moyen de 92.47%. Cette approche améliore la précision de la reconnaissance des plaques et se montre prometteuse pour des applications réelles telles que la surveillance du trafic, la gestion des péages et du stationnement.

1.6.3 Reconnaissance optique des caractères OCR

Kashyap [37] a développé un système de reconnaissance automatique des plaques d'immatriculation (ANPR) basé sur des techniques de traitement d'image et OCR pour extraire et reconnaître les caractères des plaques. L'évaluation a montré une précision de 82.6%, mais cette performance reste insuffisante pour certaines applications critiques comme l'application des lois ou la collecte de péages.

Pechiammal [38] a proposé une méthode efficace pour l'ALPR (Automatic License Plate Recognition) en trois étapes : segmentation des caractères, reconnaissance optique (OCR) et appariement des modèles avec une base de données de caractères prédéfinie. La méthode a atteint un taux d'extraction de 85%, montrant une bonne capacité à identifier correctement les caractères des plaques.

Rehman [39] a développé une approche innovante de reconnaissance des plaques en langue pakistanaise en combinant OCR et stratégies d'appariement de modèles. Testée sur des images réelles avec divers formats de plaques, cette méthode a atteint une précision de 93%, offrant ainsi un système fiable pour les applications de sécurité et de gestion de véhicules.

1.7 Détection des places de parking

Les algorithmes de classification pour la détection de l'occupation des places de stationnement se divisent en deux grandes catégories : les méthodes basées sur les zones (patch-based) et celles basées sur l'intersection (intersection-based). Les modèles de la première catégorie extraient une portion de l'image correspondant à une place de stationnement spécifique, puis appliquent un classificateur binaire sur cette zone découpée. En revanche, les modèles basés sur l'intersection identifient les boîtes englobantes des véhicules détectés et déterminent l'état d'occupation en comparant la zone d'intersection entre les voitures et les places de stationnement à un seuil prédéfini.

Dans cette section, nous présentons ces deux approches pour la détection de l'occupation des places de stationnement, en mettant également en avant les techniques de prétraitement couramment utilisées pour améliorer la qualité des prédictions dans des tâches similaires [20].

1.7.1 Patch-based methods

Les méthodes basées sur les zones (patch-based) pour la détection de l'occupation des places de stationnement reposent principalement sur les réseaux de neurones convolutionnels (CNN). Plusieurs architectures classiques ont été explorées :

- **AlexNet et mAlexNet** : Ces modèles utilisent plusieurs couches convolutionnelles et des couches entièrement connectées, avec des fonctions d'activation ReLU pour une meilleure convergence. mAlexNet, une version simplifiée, a réduit le surapprentissage tout en améliorant les prédictions.
- **CarNet** : Conçu spécifiquement pour cette tâche, CarNet intègre des convolutions dilatées pour capter des caractéristiques à grande échelle, surpassant AlexNet.
- **Détection Contrastive** : Ce modèle adopte une approche différente avec un réseau de transformation spatiale (STN) et un réseau d'extraction de caractéristiques contrastives (CFEN), afin de gérer l'occlusion entre objets et les variations de perspective.
- **VGG et ResNet** : Ces architectures plus profondes offrent de meilleures performances grâce à une plus grande capacité de modélisation, ResNet résolvant le problème de gradient évanescent par des connexions résiduelles.
- **EfficientNet** : Développé via la recherche automatisée d'architecture neuronale, EfficientNet propose un équilibre optimal entre la profondeur, la largeur et la résolution du réseau, offrant une excellente capacité de généralisation.
- **Vision Transformers (ViT)** : Ces modèles récents exploitent des encodeurs de transformers pour analyser les relations entre différentes régions d'une image, mais nécessitent de grands ensembles de données et une sélection minutieuse des hyperparamètres [20].

En résumé, les modèles patch-based dominent les approches de détection d'occupation des places de stationnement. Si les CNN classiques ont prouvé leur efficacité, des modèles plus récents comme EfficientNet et ViT présentent un fort potentiel qui mérite d'être exploré davantage, tant en termes de performance prédictive que de complexité computationnelle.

1.7.2 Intersection-based methods

Les méthodes basées sur l'intersection (intersection-based) pour la détection de l'occupation des places de stationnement se concentrent sur l'extraction des boîtes englobantes des véhicules détectés. Plusieurs approches ont été explorées :

- **Faster R-CNN (F-RCNN)** : Ce modèle utilise une région de proposition de réseau (RPN) pour générer des propositions de régions candidates, suivie d'une classification et d'un affinage des boîtes englobantes. Il offre des résultats précis mais peut être coûteux en calcul.
- **YOLO (You Only Look Once)** : YOLO traite l'image entière en un seul passage à travers le réseau, ce qui le rend extrêmement rapide tout en maintenant une bonne précision, particulièrement adapté aux systèmes en temps réel.
- **SSD (Single Shot MultiBox Detector)** : Similaire à YOLO, SSD divise l'image en grilles et prédit les boîtes englobantes directement à partir de chaque cellule de la grille, équilibrant rapidité et précision. [20]

En résumé, les méthodes intersection-based exploitent les modèles de détection d'objets pour identifier les véhicules dans les parkings, puis comparent l'intersection entre les boîtes détectées et les places de stationnement afin de déterminer l'occupation. Ces méthodes présentent un bon équilibre entre précision et efficacité, avec des modèles comme YOLO et SSD adaptés aux applications en temps réel.

1.8 Conclusion

À travers cette revue de la littérature, ce chapitre a présenté les principales approches de reconnaissance automatique des plaques d'immatriculation, mettant en évidence l'évolution des techniques, des méthodes classiques basées sur l'OCR aux modèles d'apprentissage profond comme les CNN et RNN. Malgré les avancées, des défis subsistent, notamment la gestion des plaques déformées et des variations de formats. Ces travaux constituent une base solide pour le développement de notre propre système, en visant une meilleure précision et robustesse.

Chapitre 2 Conception du système

2.1 Introduction

Ce chapitre vise à définir l'architecture générale du système, les technologies et outils adoptés, ainsi que la modélisation des différents composants. L'objectif principal de cette conception est d'assurer un système efficace, précis et robuste, capable de détecter et de reconnaître automatiquement les plaques d'immatriculation et les places de stationnement disponibles. Pour cela, une approche basée sur le deep learning est mise en place, exploitant des modèles avancés comme YOLO pour la détection d'objets et PaddleOCR pour l'extraction et la reconnaissance des caractères.

2.2 Objectifs du système

L'objectif principal de ce projet est de développer un système intelligent basé sur l'apprentissage profond, capable de détecter les places de parking ainsi que les plaques d'immatriculation, et de reconnaître les caractères qu'elles contiennent. Pour cela, des techniques avancées telles que YOLO pour la détection d'objets et PaddleOCR pour la reconnaissance des caractères ont été exploitées. Le système vise à offrir une solution précise, évolutive et sécurisée, garantissant une surveillance en temps réel et améliorant l'expérience utilisateur.

Après avoir défini les objectifs du système, il est essentiel de concevoir une architecture globale qui intègre les différentes composantes, de la détection des plaques à la gestion des places de parking.

2.3 Architecture de système

La conception d'un système intelligent de gestion de parking repose sur une architecture bien définie, intégrant plusieurs modules interconnectés. Cette section détaille l'architecture globale du système, en mettant en avant les différentes étapes du processus, depuis l'acquisition des images jusqu'à la reconnaissance des plaques d'immatriculation et la détection des places disponibles.

2.3.1 Partie détection et reconnaissance de plaques

L'architecture du système concernant la partie de détection des plaques d'immatriculation proposée repose sur deux phases principales : la **phase de détection** et la **phase de reconnaissance**. Ces deux étapes sont interconnectées afin d'assurer une extraction efficace et une reconnaissance précise des plaques d'immatriculation.

Phase	Description
Phase de détection	Entrée de l'image : Image brute fournie au modèle. Détection de la plaque : Identification de la plaque YOLOv11 . Extraction de la plaque : Découpage de la plaque pour la phase suivante.
Phase de reconnaissance	Segmentation : Ajustement éventuel pour optimiser la lecture. Reconnaissance des caractères : Extraction du texte OCR . Sortie du système : Restitution du texte pour exploitation.

TABLE 2.1 – Phases du système de reconnaissance de plaques

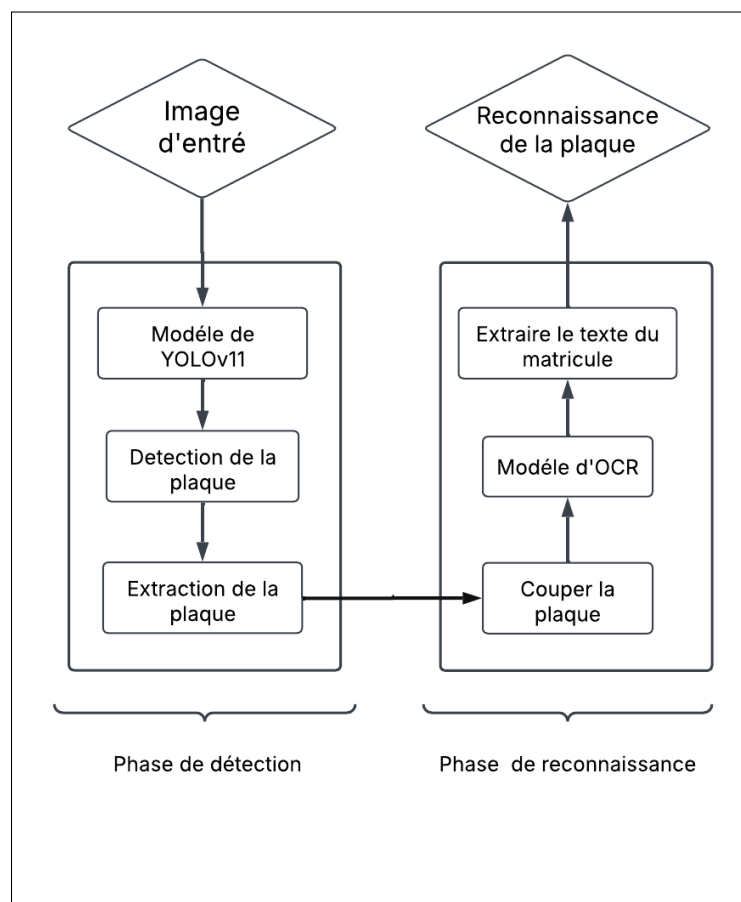


FIGURE 2.1 – Schéma d'Architecture du système

2.3.2 Partie détection des places

Cette étape consiste à détecter les places de parking à partir des images en entrée. Pour cela, le modèle YOLOv11 a été utilisé. Le système analyse l'image, identifie les véhicules et aussi fait la localisation des places, ensuite détermine également leur état : occupée ou libre. Cette information est déduite en vérifiant la présence ou l'absence d'un véhicule au sein des emplacements détectés.



Une fois l'architecture définie, l'approche méthodologique adoptée pour la mise en œuvre du système doit être détaillée. Cette approche repose sur deux axes principaux : la détection et la reconnaissance des plaques d'immatriculation, ainsi que la détection des places de parking.

2.4 Approche proposée pour la conception du système

Cette section décrit les différentes étapes de l'approche proposée, en mettant l'accent sur les choix technologiques et les stratégies d'optimisation employées pour assurer la performance et la robustesse du système.

2.4.1 Détection et reconnaissance des plaque d'immatriculation

Cette partie présente le processus de détection des plaques d'immatriculation, allant de l'installation et la configuration de YOLOv11 à l'entraînement et l'évaluation du modèle, en mettant en avant les choix méthodologiques adoptés.

2.4.1.1 Modèle de détection YOLOv11

Pour la tâche de détection des plaques d'immatriculation, nous avons opté pour le modèle YOLOv11 en raison de ses performances remarquables en matière de détection d'objets. Ce modèle se distingue par sa capacité à identifier rapidement et avec précision les plaques dans des environnements variés, même en présence de conditions complexes telles que des angles de vue différents, des variations de luminosité ou des obstructions partielles. L'architecture optimisée de YOLOv11 permet un traitement en temps réel, ce qui est essentiel pour une application de gestion intelligente de parking. Ces atouts en font un choix pertinent pour notre système.

Pseudo code du modèle Le pseudo-code suivant résume les principales étapes de la détection des plaques d'immatriculation avec YOLOv11 :

Algorithm 1 YOLOv11 License Plate Detection Process

```

1: Début
2: 1. Préparation des données :
3:   Charger le dataset brut (images + annotations XML)
4:   Appliquer l'augmentation de données pour enrichir le dataset
5:   Convertir les annotations au format YOLO (fichiers TXT)
6: 2. Division du dataset :
7:   Diviser les données en ensembles d'entraînement, validation et test
8: 3. Configuration du modèle :
9:   Cloner le dépôt YOLOv11 et installer les dépendances
10:  Configurer le fichier dataset.yaml
11: 4. Entraînement du modèle :
12:  Charger les images et annotations d'entraînement
13:  Définir les hyperparamètres : résolution 640x640, batch size = 32, époques = 100
14:  Exécuter le processus d'entraînement
15: 5. Évaluation du modèle :
16:  Charger l'ensemble de test
17:  Prédire les boîtes englobantes sur les images test
18:  Calculer les métriques : précision, rappel et mAP
19:  Tracer la courbe PR pour évaluer le compromis précision/rappel
20: 6. Sauvegarde des résultats :
21:  Exporter le modèle entraîné et les métriques d'évaluation
22: Fin

```

Préparation et Annotation des Données Le dataset utilisé dans ce projet provient de Kaggle, intitulé *Car Plate Detection* [40]. Il se compose de 433 images de véhicules avec leurs plaques d'immatriculation visibles. Afin d'enrichir ce dataset, des techniques d'augmentation de données ont été appliquées à l'aide de la plateforme Roboflow, portant le total à 700 images. Chaque image est accompagnée d'un fichier XML au format Pascal VOC contenant les annotations des plaques d'immatriculation, qui ont ensuite été converties au format *YOLO* pour assurer la compatibilité avec *YOLOv11*. Cette conversion transforme chaque fichier *XML* en un fichier *TXT* contenant les identifiants de classe et les coordonnées normalisées des boîtes englobantes.

Pour une préparation optimale du dataset, les données ont été divisées en trois ensembles, 70% pour l'entraînement ; utilisé pour l'apprentissage du modèle, 20% pour l'ensemble de validation qui es utilisé pour l'ajustement des hyeparamètres et pour éviter le surapprentissage, et un pourcentage de 10% pour les données de test qui on été utilisé pour évaluer les performances finales du modèle sur des données non vues.

Cette structure garantit une bonne gestion des données et une préparation adéquate pour le processus d'entraînement de YOLOv11. La figure ci-dessous illustre un exemple d'image avec son annotation initiale au format *XML* et sa conversion au format *TXT* :

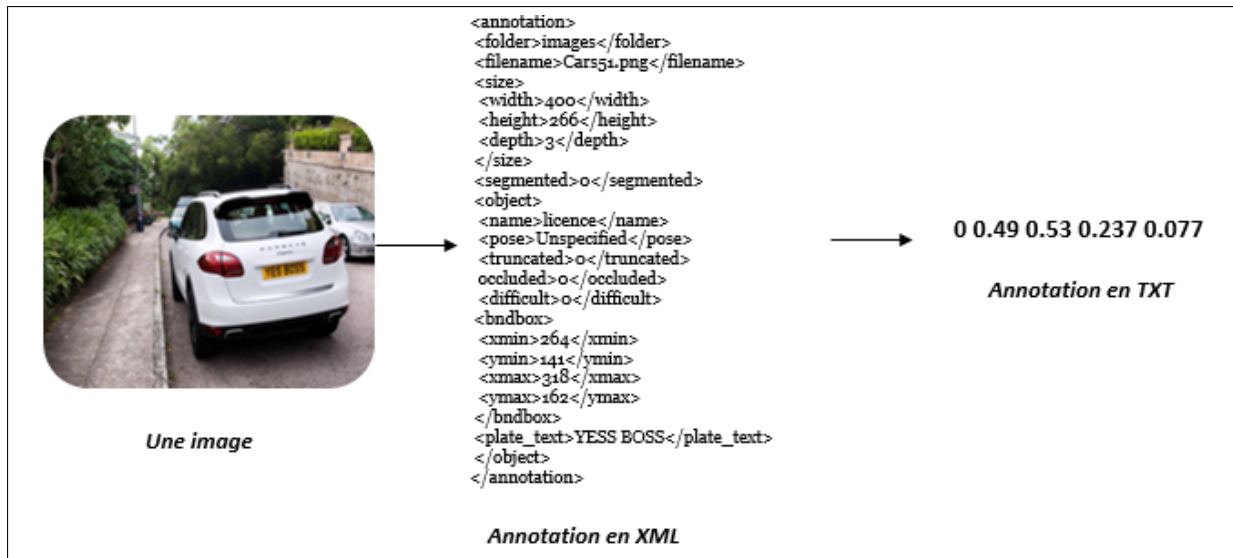


FIGURE 2.2 – Exemple d’une image du dataset avec ses annotations

Installation et Configuration de YOLOv11 YOLOv11 a été installé et configuré à partir du dépôt officiel GitHub [41]. Un fichier `dataset.yaml` a été créé pour définir les paramètres du jeu de données, notamment les chemins d’accès, le nombre de classes et leurs noms, permettant ainsi au modèle de comprendre la structure des données utilisées. L’entraînement a été réalisé en utilisant une taille des images qui est 640x640 pixels, un batch size égal à 32 et un total de nombre d’époques de 100. Ces paramètres ont permis au modèle d’apprendre efficacement les caractéristiques des plaques d’immatriculation, tout en assurant une convergence stable pendant l’entraînement.

Détection et extraction de la plaque La détection et l’extraction de la plaque d’immatriculation à l’aide de YOLOv11 suivent un pipeline en trois étapes. Tout d’abord, une image contenant un véhicule est fournie en entrée du modèle. Celui-ci redimensionne automatiquement l’image à une résolution standard de 640x640 pixels pour optimiser le traitement, puis prédit la position précise de la plaque en générant une boîte englobante (bounding box) autour de la région cible. Enfin, les coordonnées de cette boîte permettent d’extraire et d’isoler la plaque comme région d’intérêt (ROI), préparant ainsi le terrain pour les phases ultérieures de reconnaissance des caractères par OCR.



FIGURE 2.3 – Les étapes de détection d’une plaque

2.4.1.2 Modèle de reconnaissance PaddleOCR

Pour la tâche de reconnaissance des caractères (OCR), nous avons choisi d'utiliser PaddleOCR, une bibliothèque puissante et une boîte à outils OCR flexible dédiée à l'extraction de texte à partir d'images. PaddleOCR offre plusieurs avantages, notamment sa capacité à gérer des textes dans différentes langues et écritures, ce qui en fait un choix pertinent pour la reconnaissance des plaques d'immatriculation. De plus, son architecture modulaire permet une intégration aisée avec notre système de détection de plaques basé sur YOLOv11. Grâce à ses modèles pré-entraînés [42] et à sa prise en charge des environnements de calcul optimisés, PaddleOCR garantit des résultats précis et rapides, même dans des conditions variées telles que des angles de vue différents ou des éclairages complexes.

Pseudo code de l'OCR Le pseudo-code ci-dessous décrit les étapes principales du processus de reconnaissance des caractères à l'aide de PaddleOCR, appliqué sur les plaques d'immatriculation détectées :

Algorithm 2 Pseudo-code du modèle PaddleOCR (modèle pré-entraîné utilisé pour l'extraction de texte des plaques d'immatriculation)

```

1: Initialisation :
2: Importer PaddleOCR et les bibliothèques nécessaires
3: Charger le modèle pré-entraîné de PaddleOCR
4: Préparation des données :
5: for chaque image dans le dataset annoté do
6:     Charger l'image
7: end for
8: Utilisation du modèle pré-entraîné pour l'extraction de texte :
9: for chaque image dans le dataset do
10:     Appliquer le modèle pré-entraîné de PaddleOCR pour extraire le texte des plaques
        d'immatriculation
11: end for
12: Évaluation du modèle :
13: for chaque image dans l'ensemble de test annoté do
14:     Appliquer le modèle pour extraire le texte
15:     Comparer la sortie avec l'annotation réelle de la plaque d'immatriculation
16:     Calculer les métriques : Taux d'Erreur des Caractères CER et des mots WER
17: end for
18: Résultats :
19: Afficher la précision d'extraction du texte en fonction des annotations

```

Extraction et reconnaissance du texte Nous avons utilisé *PaddleOCR* comme solution pré-entraînée, sans aucune phase d'apprentissage supplémentaire. Il a été directement intégré dans notre système et combiné avec notre modèle YOLO afin d'extraire et lire le texte des plaques détectées.

Une fois la reconnaissance des caractères effectuée avec PaddleOCR, la sortie est une chaîne de caractères représentant le matricule détecté. Toutefois, ce texte brut peut contenir des erreurs dues aux variations d'éclairage, aux distorsions ou à la qualité de l'image. Pour y remédier, un post-traitement est appliqué afin de vérifier la validité du matricule.

Prétraitement appliqué sur les images de plaques Avant d'envoyer l'image détectée vers le modèle Paddle OCR, plusieurs opérations de prétraitement sont appliquées pour améliorer la qualité et la lisibilité des caractères :

1. **Redimensionnement** (optionnel)

Un redimensionnement est appliqué uniquement lorsque la détection YOLO n'est pas optimale. L'image est redimensionnée à une taille fixe de 300×100 pixels pour uniformiser les dimensions.

$$I_{resized}(x, y) = I\left(\frac{x \cdot w_{orig}}{w_{new}}, \frac{y \cdot h_{orig}}{h_{new}}\right) \quad (2.1)$$

où w_{orig}, h_{orig} sont les dimensions originales et w_{new}, h_{new} les nouvelles dimensions.



FIGURE 2.4 – Résultat après redimensionnement

2. **Conversion en niveaux de gris**

L'image couleur est convertie en niveaux de gris pour simplifier le traitement ultérieur, en utilisant la moyenne pondérée des canaux RGB :

$$I_{gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B \quad (2.2)$$



FIGURE 2.5 – Image après conversion en niveaux de gris

3. **Réduction du bruit**

Un filtre médian est appliqué pour supprimer le bruit tout en préservant les contours :

$$I_{denoised}(x, y) = \text{median}\{I_{CLAHE}(x + i, y + j)\}, \quad i, j \in [-k, k] \quad (2.3)$$

avec $k = 3$ définissant la taille du noyau (kernel).



FIGURE 2.6 – Image après réduction de bruit

4. **Binarisation adaptative**

Seuillage adaptatif gaussien pour séparer le texte du fond :

$$I_{binary}(x, y) = \begin{cases} 255 & \text{si } I_{denoised}(x, y) > \mu(x, y) + C \\ 0 & \text{sinon} \end{cases} \quad (2.4)$$

où $\mu(x, y)$ est la moyenne pondérée gaussienne locale, $C = 2$ est une constante, et la taille de voisinage est de 11×11 pixels.

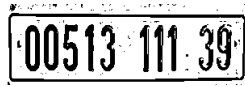


FIGURE 2.7 – Résultat après binarisation adaptative

2.4.2 Détection des places de parking

Dans cette section, nous allons expliquer l'architecture et les paramètres de notre modèle de détection.

2.4.2.1 Modèle YOLOv11 utilisé

Pour la détection des places de parking, nous avons également choisi d'utiliser le modèle YOLOv11 en raison de sa capacité à fournir des prédictions rapides et précises. En exploitant les poids optimaux obtenus après l'entraînement, YOLOv11 permet d'identifier efficacement les emplacements de stationnement disponibles, même dans des environnements complexes présentant des variations d'éclairage, des angles de vue différents ou des véhicules partiellement obstruant les places. Cette étape est essentielle pour le développement du système intelligent de gestion de parking, en offrant une base solide pour la prédiction de la disponibilité des places.

Entraînement du modèle YOLOv11 L'entraînement du modèle YOLOv11 a été réalisé afin d'optimiser la détection des places de parking. En utilisant un jeu de données annoté, le modèle apprend à reconnaître les différentes configurations de stationnement, distinguant les places occupées des places vides avec précision.

Préparation des données Le dataset utilisé provient de la plateforme Roboflow [43] et comprend 600 images, réparties en trois ensembles : 80% pour l'entraînement, 20% pour la validation et 10% pour le test. Les images sont annotées au format YOLOv11 (*txt*), accompagnées d'un fichier de configuration au format YAML, facilitant l'intégration dans le processus d'entraînement.

Exemple d'une image de dataset : La Figure 2.9 présente un échantillon typique du jeu de données d'entraînement avec ses annotations au format YOLOv11

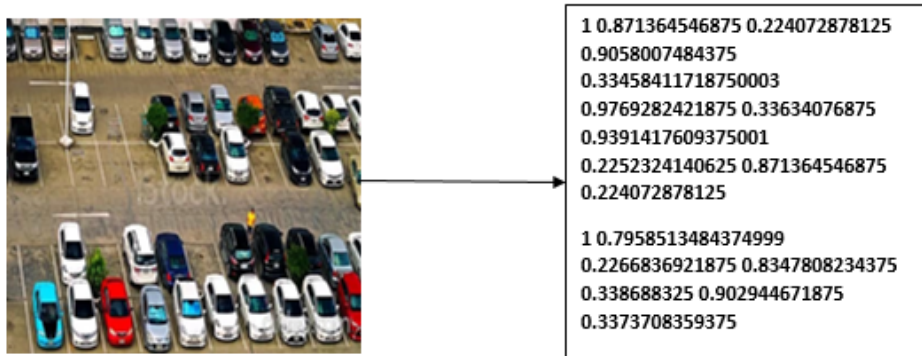


FIGURE 2.8 – Exemple sur les données d'entrainement du modèle de détection des places

Installation et Configuration du modèle Avant l'entraînement, le modèle YOLOv11 a été importé [41] et initialisé avec ses poids pré-entraînés. Ensuite, plusieurs hyperparamètres clés ont été configurés, notamment le nombre d'époques fixé à 100 et une taille d'image de 640x640 pixels. L'entraînement a été réalisé à l'aide du fichier de configuration `data.yaml` fourni avec le dataset, spécifiant les classes et les chemins d'accès aux données. Les résultats ont été sauvegardés dans un répertoire dédié, permettant d'analyser les performances du modèle et de sélectionner les meilleurs poids pour la détection des places de parking.

Détection et classification des places Une fois le modèle YOLOv11 entraîné, il a été utilisé pour détecter les places de stationnement dans les images du parking. Chaque place identifiée a ensuite été classée en deux catégories : occupée ou libre, en fonction de la présence d'un véhicule. Cette classification permet de fournir un état en temps réel de l'occupation du parking. La Figure ci-dessous illustre les différentes étapes de notre système de détection des places de parking : depuis l'acquisition de l'image jusqu'à la classification finale des spots, en passant par le découpage en grille, la prédiction des bounding boxes et leur filtrage.

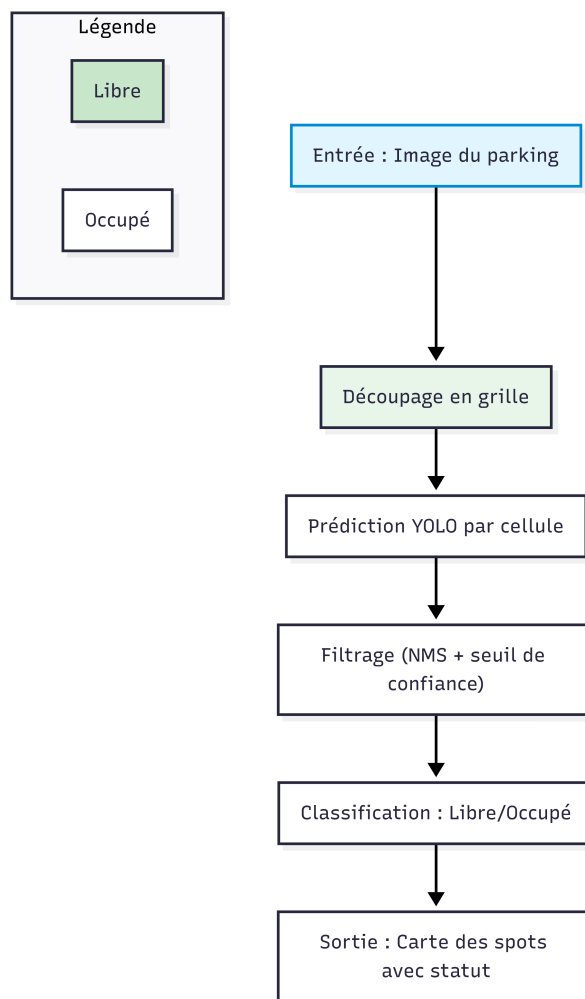


FIGURE 2.9 – Le processus de détection des places

2.5 Conclusion

Dans ce chapitre, nous avons détaillé l'architecture et la conception du système de reconnaissance automatique des plaques d'immatriculation. Nous avons commencé par la phase de détection des plaques, où le modèle YOLOv11 a été entraîné et utilisé pour identifier et extraire les plaques à partir des images d'entrée. Ensuite, nous avons abordé la reconnaissance des caractères en exploitant PaddleOCR, en appliquant un prétraitement des images, une segmentation des caractères et une extraction du texte. Enfin, une structuration du matricule a été effectuée pour garantir un format cohérent et exploitable. L'ensemble de ces étapes constitue un pipeline robuste permettant une identification fiable des plaques d'immatriculation.

Chapitre 3 Implémentation du système

3.1 Introduction

L'implémentation d'un système de reconnaissance des plaques d'immatriculation repose sur plusieurs étapes clés allant de la préparation de l'environnement de travail jusqu'à l'analyse des résultats obtenus. Ce chapitre décrit en détail le processus de développement du système, en mettant en avant les outils, les technologies et les approches adoptées.

Dans un premier temps, nous présenterons l'environnement de travail, incluant les plateformes utilisées pour l'entraînement des modèles et les ressources matérielles disponibles. Ensuite, nous détaillerons les choix technologiques en termes de langage de programmation et de bibliothèques, justifiant leur pertinence pour le traitement des images et la reconnaissance de texte.

Par la suite, une vue d'ensemble du système sera fournie, illustrant les différentes composantes du pipeline de traitement : de la détection des plaques à l'extraction des caractères. Enfin, nous présenterons les résultats obtenus, accompagnés d'une analyse approfondie permettant de mettre en évidence les performances du système et d'identifier les axes d'amélioration potentiels.

3.2 Environnement de développement

Le développement des modèles de détection et de reconnaissance des plaques d'immatriculation a été réalisé sur **Google Colab**, permettant l'accès à des ressources de calcul puissantes, notamment des **GPU Tesla T4** pour l'entraînement des modèles de deep learning. En parallèle, le développement de l'application a été effectué localement sur un ordinateur personnel équipé des caractéristiques suivantes :

- **Système d'exploitation** : Windows 10 Pro
- **Processeur (CPU)** : Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz
- **Mémoire vive (RAM)** : 8.00 GB
- **Type de système** : 64-bit operating system, x64-based processor
- **Carte graphique (GPU)** :
- **Environnement de développement** : Visual Studio Code (VS Code)

Pour assurer la gestion du code source et faciliter le travail collaboratif, le projet a été versionné et déployé sur la plateforme **GitHub**. Cela a permis de centraliser les différentes contributions, de suivre les modifications, et de maintenir un historique clair du développement.

3.3 Technologies et bibliothèques utilisées

Dans cette section, nous allons présenter les technologies et bibliothèques utilisées pour le développement, l'entraînement des modèles et le traitement des données.

3.3.1 Language de programmation et outils utilisés

Python Python est un langage de programmation interprété, orienté objet et de haut niveau, avec une sémantique dynamique. Ses structures de données intégrées de haut niveau, combinées à un typage dynamique et à un couplage dynamique, le rendent très attrayant pour le développement rapide d'applications, ainsi que pour une utilisation comme langage de script ou de liaison afin de connecter différents composants existants. Sa syntaxe simple et facile à apprendre met l'accent sur la lisibilité, réduisant ainsi le coût de maintenance des programmes. Python prend en charge les modules et les packages, ce qui favorise la modularité des programmes et la réutilisation du code. L'interpréteur Python et sa bibliothèque standard étendue sont disponibles sous forme de code source ou de binaire gratuitement pour toutes les principales plateformes, et peuvent être librement distribués [44].

Google Collab Google Colab, abréviation de Google Colaboratory, est un environnement de développement interactif qui permet d'exécuter du code Python directement dans le cloud. Il est basé sur Jupyter Notebook et offre une interface conviviale pour le développement de projets de machine learning et de deep learning. L'un de ses principaux avantages est qu'il ne nécessite aucune configuration locale, tout le travail se fait via le navigateur, éliminant ainsi le besoin d'installer des logiciels spécifiques sur la machine de l'utilisateur. De plus, Colab donne accès gratuitement à des ressources de calcul puissantes telles que les GPU et les TPU, ce qui le rend particulièrement adapté aux tâches nécessitant de grandes capacités de traitement [45].

Visual Studio Code (VS Code) Visual Studio Code (VS Code) est un éditeur de code source moderne et léger, développé par Microsoft. Il prend en charge de nombreux langages de programmation et offre une intégration fluide avec des outils de développement, notamment pour le Python. Grâce à ses fonctionnalités avancées, comme l'autocomplétion intelligente, le débogage intégré et la gestion de versions via Git, VS Code est largement utilisé dans le domaine de la data science et du développement d'applications.

L'un des principaux avantages de VS Code est sa flexibilité, grâce à une vaste bibliothèque d'extensions permettant d'adapter l'environnement aux besoins spécifiques du projet. Dans le cadre de ce travail, VS Code a été utilisé pour développer et tester l'application en local avant son intégration complète [46].

3.4 Bibliothèques utilisées

Dans ce projet, nous avons utilisé plusieurs bibliothèques essentielles pour l'entraînement et l'évaluation des modèles d'apprentissage profond. Ces bibliothèques couvrent divers aspects tels que la détection d'objets, l'entraînement de modèles, le traitement d'images, l'affichage des résultats et bien plus encore. Voici une présentation détaillée des principales bibliothèques utilisées :

Ultralytics YOLO Ultralytics YOLO est une implémentation moderne et optimisée des modèles YOLO (You Only Look Once), conçue pour la détection en temps réel d'objets dans les images et vidéos. Cette bibliothèque fournit des outils conviviaux permettant d'entraîner, d'évaluer et de déployer des modèles de détection avec un minimum de configuration. Elle prend en charge des améliorations avancées telles que l'optimisation des hyperparamètres et le support des accélérateurs matériels [47].

TensorFlow TensorFlow est une bibliothèque open-source développée par Google pour le machine learning et le deep learning. Elle offre une interface flexible permettant de construire et d'entraîner des réseaux de neurones profonds sur diverses architectures matérielles, notamment les CPU, GPU et TPU. TensorFlow est particulièrement adapté aux applications impliquant la vision par ordinateur, le traitement du langage naturel et l'apprentissage par renforcement [48].

PyTorch PyTorch, développé par Meta AI, est une bibliothèque très populaire pour le deep learning, largement utilisée en recherche et en production. Contrairement à TensorFlow, PyTorch repose sur une approche de graphes de calcul dynamiques, ce qui facilite le débogage et l'expérimentation. Il est souvent privilégié pour le prototypage rapide et l'entraînement de modèles neuronaux complexes, notamment dans les tâches de classification, de segmentation et de reconnaissance d'objets [49].

OpenCV OpenCV (Open Source Computer Vision Library) est une bibliothèque open-source spécialisée en vision par ordinateur. Elle fournit des outils performants pour la manipulation et l'analyse d'images, y compris la détection de contours, la transformation d'images et l'extraction de caractéristiques visuelles. Dans notre projet, OpenCV a été utilisé pour le prétraitement des images, notamment l'amélioration de la qualité des plaques d'immatriculation avant leur reconnaissance [50].

Matplotlib Matplotlib est une bibliothèque de visualisation en Python permettant de générer des graphiques 2D et 3D. Elle est particulièrement utilisée pour représenter graphiquement les résultats des modèles de machine learning sous forme de courbes de performance, d'histogrammes et de matrices de confusion. Dans notre projet, nous avons utilisé Matplotlib pour visualiser l'évolution des performances des modèles entraînés et pour représenter les prédictions obtenues [51].

NumPy NumPy (Numerical Python) est une bibliothèque fondamentale pour le calcul scientifique en Python. Elle fournit un support efficace pour la manipulation de tableaux multidimensionnels et l'exécution d'opérations mathématiques avancées. Elle est souvent

utilisée comme base pour d'autres bibliothèques, telles que TensorFlow et PyTorch, qui s'appuient sur ses structures de données optimisées [52].

Pillow (PIL) Pillow est une bibliothèque de traitement d'images dérivée du projet PIL (Python Imaging Library). Elle permet d'effectuer diverses transformations sur les images, notamment le redimensionnement, la conversion de formats et l'application de filtres. Dans notre projet, nous avons utilisé Pillow pour normaliser et améliorer la qualité des images de plaques d'immatriculation avant leur passage dans le modèle de reconnaissance de caractères [53].

PaddleOCR PaddleOCR est une bibliothèque open-source spécialisée dans la reconnaissance optique de caractères (OCR). Développée par PaddlePaddle, elle offre des modèles optimisés pour extraire du texte à partir d'images. Elle est particulièrement efficace pour reconnaître des caractères dans des conditions variées, telles que des textes flous ou mal alignés. Bien que nous ayons envisagé son utilisation, nous avons finalement opté pour d'autres solutions pour la reconnaissance des plaques d'immatriculation [54].

3.5 Vue d'ensemble du système

3.6 Cas d'utilisation du système

3.6.1 Exemple de détection et reconnaissance de matricule

3.6.2 Exemple de détection de places d'un parking

3.7 Analyse des résultats

Dans cette section, nous présentons l'évaluation des différents modèles constituant notre système. Chaque modèle a été testé à l'aide d'un ensemble de données dédié afin de mesurer sa performance à travers des métriques pertinentes telles que la précision, le rappel ...ect, selon le cas. Pour évaluer les performances de nos modèles, nous avons utilisé les métriques suivantes, en fonction de la nature de chaque tâche :

- **Précision (Precision)** :

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Rappel (Recall)** :

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score** :

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **mAP@0.5 (mean Average Precision)** :

$$\text{mAP@0.5} = \frac{1}{N} \sum_{i=1}^N AP_i$$

- **CER (Character Error Rate)** : utilisé pour évaluer la reconnaissance de texte caractère par caractère. Il se définit comme :

$$\text{CER} = \frac{S + D + I}{N}$$

où :

- S : substitutions,
- D : suppressions,
- I : insertions,
- N : nombre total de caractères dans la transcription de référence.
- **WER (Word Error Rate)** : mesure les erreurs au niveau des mots :

$$\text{WER} = \frac{S + D + I}{N}$$

Ici, S , D , I sont calculés au niveau des mots, et N est le nombre total de mots dans la transcription de référence.

3.7.1 Résultats du modèle de détection des places

Le modèle **YOLOv11** a été utilisé pour la détection automatique des plaques d'immatriculation. Pour évaluer ses performances, un ensemble de test constitué d'images de véhicules, a été utilisé. Le tableau ci-dessous résume les performances finales du modèle. Ces performances seront analysées plus en détail dans les sections suivantes.

Précision	Rappel	mAP@50	mAP@50-95	Box Loss	Class Loss
0.87	0.84	0.87	0.46	1.69	0.86

TABLE 3.1 – Résumé des performances finales du modèle

Précision, Rappel et mAP

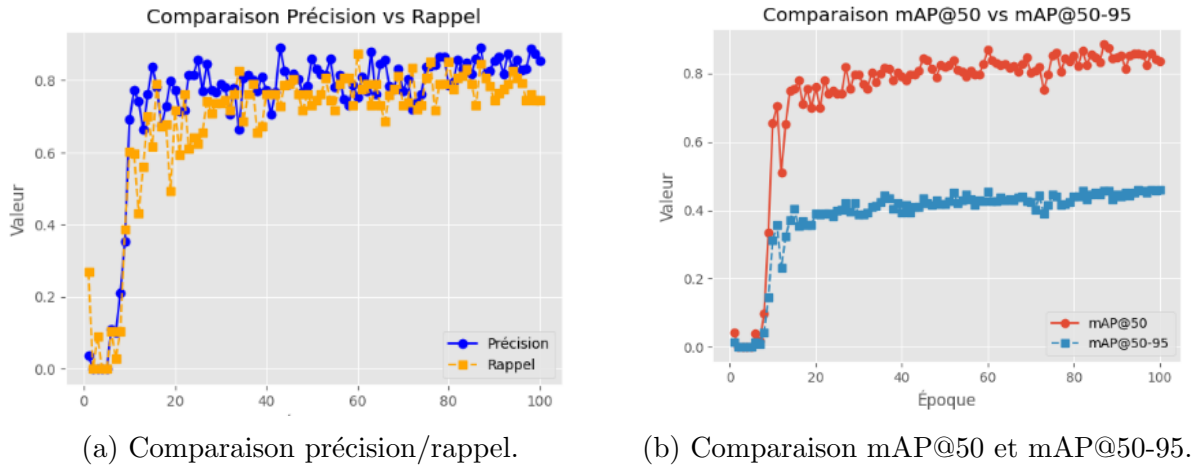


FIGURE 3.1 – Performances du modèle en termes de détection.

Compromis précision/rappel : La Figure 3.1a montre une précision élevée (≥ 0.8) mais un rappel modéré (~ 0.6), indiquant que le modèle manque certaines plaques (faux négatifs). Pour améliorer le rappel, un ajustement du seuil de confiance ou l'ajout de données complexes (e.g., plaques occultées) est recommandé.

Analyse de la mAP : La Figure 3.1b souligne une forte mAP@50 (~ 0.8) mais une mAP@50-95 faible (~ 0.4), reflétant une localisation approximative. Ceci pourrait être résolu en optimisant la taille des *bounding boxes* ou en utilisant des techniques de super-résolution pour les petites plaques.

Métriques de Localisation et Classification

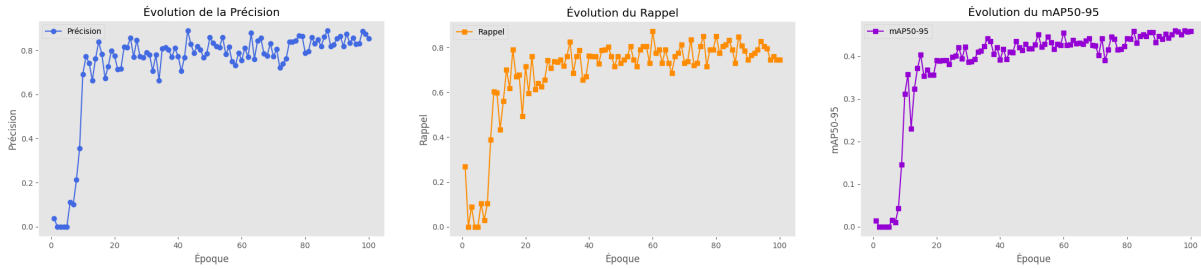


(a) Évolution des pertes de localisation (Box Loss) pendant l'entraînement et la validation. (b) Évolution des pertes de classification (CLS Loss) pendant l'entraînement et la validation.

FIGURE 3.2 – Analyse des pertes du modèle YOLOv11.

Analyse des pertes : La Figure 3.2a montre que la *Train Box Loss* diminue de manière stable, tandis que la *Validation Box Loss* présente des oscillations, suggérant une sensibilité aux variations des données de validation. La Figure 3.2b révèle un écart croissant entre les pertes d'entraînement et de validation en fin d'époques, signalant un possible surapprentissage. Pour y remédier, des techniques de régularisation (e.g., *Dropout*) ou une augmentation des données (e.g., rotations aléatoires) pourraient être appliquées.

Évolution des Métriques au Cours de l'Entraînement



(a) Précision par époque.

(b) Rappel par époque.

(c) mAP@50-95 par époque.

FIGURE 3.3 – Évolution des métriques au fil des époques.

Tendances longitudinales : Les Figures 3.3a–3.3c montrent que : **La précision** se stabilise rapidement, confirmant la fiabilité des prédictions positives, **Le rappel** stagne, nécessitant des données plus diversifiées et **La mAP@50-95** progresse lentement, ce qui pourrait justifier l'utilisation d'un *learning rate scheduler* pour affiner les poids en fin d'entraînement.

Exemple d'application : Dans cette image, nous voyons un exemple d'application du modèle YOLOv11 pour la détection de matricule. Le modèle, entraîné pour identifier des plaques d'immatriculation de véhicules, analyse l'image et localise précisément la zone où la plaque est présente. Grâce à ses performances avancées, YOLOv11 est capable de détecter et de délimiter la plaque d'immatriculation avec une grande précision, ce qui permet une reconnaissance automatique des matricules dans des contextes variés.



FIGURE 3.4 – Exemple d'application sur une image.

3.7.2 Résultats du modèle de reconnaissance des plaques

Pour la tâche de reconnaissance optique des caractères (OCR), nous avons utilisé le modèle pré-entraîné **PaddleOCR**, qui offre des performances de pointe sans nécessiter de phase d'entraînement supplémentaire. Afin d'évaluer ses performances dans notre contexte, nous avons testé ce modèle sur un ensemble d'images de plaques d'immatriculation algériennes, issues du dataset [55]. Ce jeu de données contient des images annotées de plaques algériennes réelles, offrant ainsi un cadre idéal pour une évaluation réaliste du modèle de reconnaissance.

Illustration d'une annotation Les images du dataset utilisé sont nommées de manière à contenir directement la transcription du texte de la plaque. Cela permet une extraction automatique de la vérité terrain sans avoir besoin de fichiers d'annotation séparés. Par exemple, une image nommée :

0473810543.jpg

correspond à une plaque contenant le texte **0473 105 43**, où chaque segment représente respectivement le numéro, l'année, et le code de wilaya.



FIGURE 3.5 – Exemple d'image issue du dataset.

Métriques utilisées Deux métriques principales ont été utilisées pour quantifier les performances de reconnaissance :

- **CER (Character Error Rate)** : Taux d’erreurs de caractères, mesurant le nombre total de substitutions, insertions et suppressions nécessaires pour transformer la sortie prédite en la chaîne cible, divisé par le nombre total de caractères attendus.
- **WER (Word Error Rate)** : Variante du CER à l’échelle des mots, adaptée lorsque les plaques contiennent plusieurs groupes de caractères (lettres, chiffres).

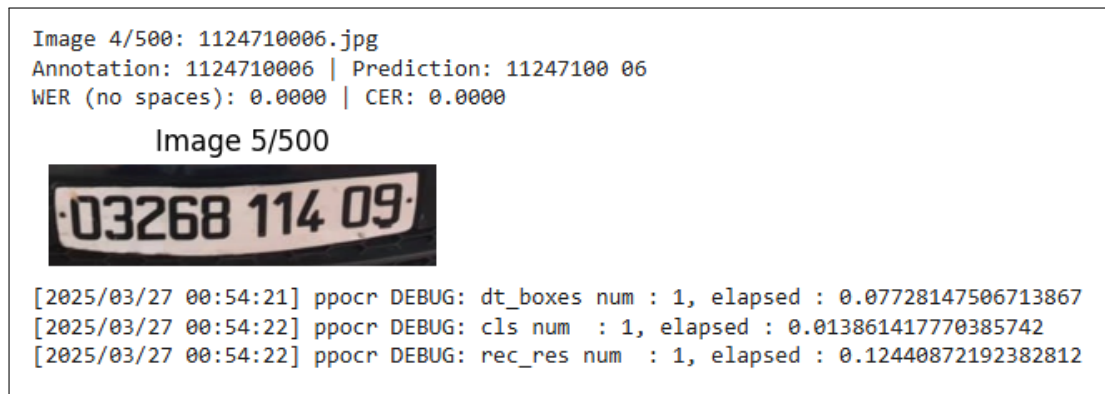


FIGURE 3.6 – Reconnaissance de matricule : Exemple et scores WER/CER



FIGURE 3.7 – Scores CER et WER moyens sur l’ensemble de test

Résultats obtenus Les évaluations montrent que le modèle PaddleOCR est capable de reconnaître correctement la majorité des plaques, avec un **CER moyen inférieur à 5%** et un **WER inférieur à 25%**. Ces résultats sont satisfaisants compte tenu de la variabilité des conditions (angle, éclairage, qualité d’image). Des erreurs subsistent notamment sur des plaques partiellement occultées ou floues.

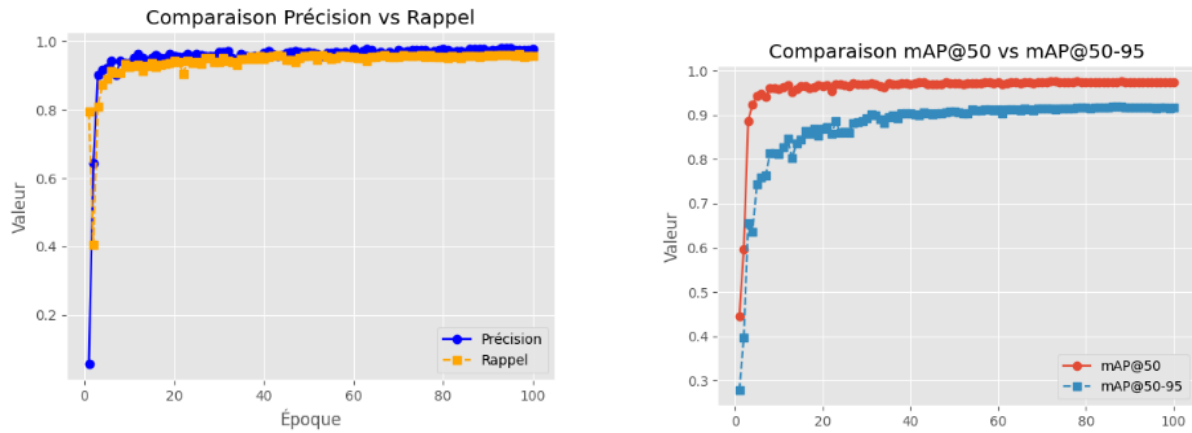
3.7.3 Résultats du modèle de détection des places

Le modèle **YOLOv11** a été entraîné spécifiquement pour la détection des places de parking. L'évaluation a été réalisée sur un ensemble de test varié. Le tableau ci-dessous résume les performances finales du modèle de détection des places de parking, mettant en évidence les principales métriques obtenues après l'entraînement.

Précision	Rappel	mAP@50	mAP@50-95	Box Loss	Class Loss
0.98	0.96	0.97	0.92	0.52	0.34

TABLE 3.2 – Performances finales du modèle de détection des places de parking

Précision, Rappel et mAP



(a) Comparaison précision/rappel.

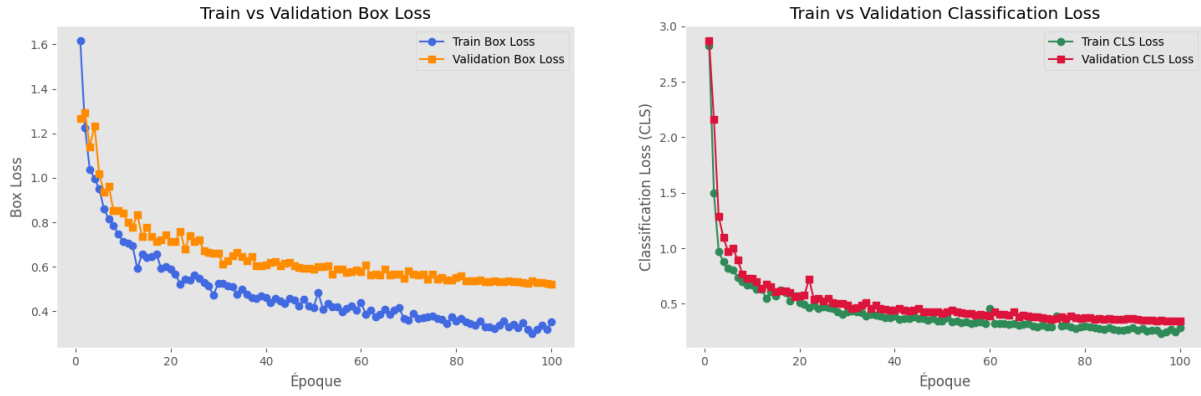
(b) Comparaison mAP@50 et mAP@50-95.

FIGURE 3.8 – Performances du modèle sur la détection des places de parking.

Compromis précision/rappel : La Figure 3.8a montre une précision excellente (≥ 0.9) et un rappel élevé (~ 0.8), indiquant que le modèle détecte efficacement les places de parking tout en minimisant les faux positifs. Ces résultats suggèrent une bonne adaptation aux variations géométriques des places.

Analyse de la mAP : La Figure 3.8b révèle une mAP@50 très élevée (0.9) et une mAP@50-95 respectable (0.7), démontrant une localisation précise des places. L'écart entre les deux métriques est réduit par rapport à la détection de plaques, ce qui s'explique par la nature plus régulière et moins variable des places de parking.

Métriques de Localisation et Classification



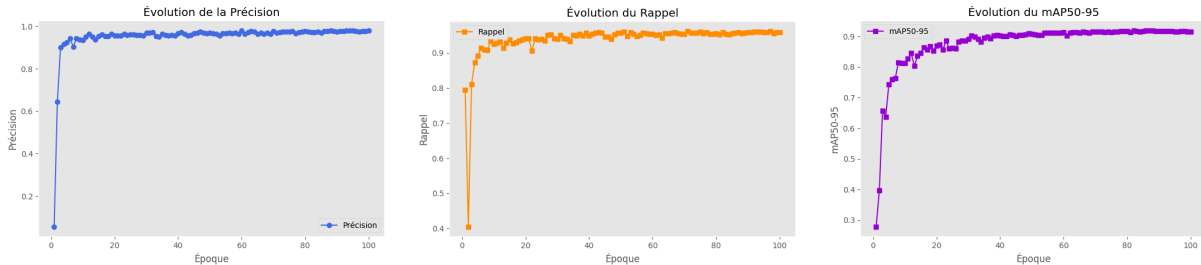
(a) Pertes de localisation (Box Loss).

(b) Pertes de classification (CLS Loss).

FIGURE 3.9 – Évolution des pertes pour la détection de places.

Analyse des pertes : La Figure 3.9a montre une convergence rapide et stable des pertes de localisation, avec un écart minime entre entraînement et validation, contrairement au cas des plaques. Cela confirme la simplicité relative de la tâche pour le modèle. La Figure 3.9b présente des pertes de classification plus basses et stables, grâce à la distinction claire entre places occupées/libres.

Évolution des Métriques



(a) Précision.

(b) Rappel.

(c) mAP@50-95.

FIGURE 3.10 – Tendances des métriques par époque.

Performances par époque : Les Figures 3.10a–3.10c indiquent que : **La précision** atteint rapidement un plateau à 0.9 dès les premières époques, **le rappel** progresse régulièrement jusqu'à 0.8, suggérant une amélioration continue sur les cas difficiles et **la mAP@50-95** montre une progression linéaire, confirmant l'apprentissage stable des caractéristiques géométriques.

Exemple d'application : Cet exemple montre l'application de YOLOv11 pour détecter les places de parking libres ou occupées. Le modèle analyse l'image du parking, identifie les véhicules présents et classe chaque place comme 'vide' ou 'occupée', facilitant ainsi la gestion intelligente du stationnement.”

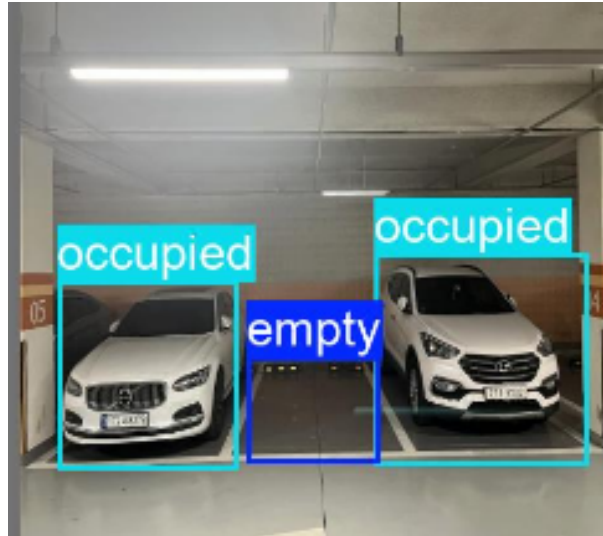


FIGURE 3.11 – Exemple d'application de détection des places.

3.8 Discussion

Nous proposons ici une réflexion critique sur les résultats expérimentaux, en mettant en lumière les atouts, les limites rencontrées et les perspectives d'évolution du système

3.8.1 Étude des performances obtenues

Les performances globales du système sont encourageantes, avec des résultats satisfaisants dans les différentes étapes du pipeline :

- **Détection des plaques** : Le modèle YOLOv11 atteint une précision d'environ **87%**, ce qui montre sa capacité à détecter efficacement les plaques dans des environnements variés.
- **Détection des places de stationnement** : Cette tâche obtient une précision élevée, avoisinant les **97%**, démontrant la robustesse du système même dans des scènes complexes.
- **Reconnaissance OCR** : Le taux d'erreur de caractères (CER) est de **5%**, ce qui indique une bonne qualité de transcription des plaques, malgré quelques erreurs sur certains cas particuliers.

3.8.2 Limites Identifiées

Malgré les bons résultats, certaines limites subsistent :

- **Erreurs de lecture** sur les plaques partiellement illisibles, abîmées ou couvertes de saletés.
- **Baisse de performance** dans des conditions défavorables telles que des parkings mal éclairés, une forte réflexion lumineuse ou des images floues.
- **Sensibilité à la rotation** : les plaques inclinées ou fortement penchées sont parfois mal détectées ou mal reconnues.

3.8.3 Pistes d'Amélioration

Pour accroître la robustesse et la généralisation du système, plusieurs axes d'amélioration sont envisagés :

- **Renforcement des données d'entraînement** :
 - Intégrer davantage d'images sous différents angles et dans des conditions variées (pluie, nuit, faible luminosité, etc.).
 - Ajouter des exemples de plaques partiellement visibles ou fortement dégradées.
- **Optimisation du modèle** :
 - Intégrer un module de correction de rotation ou de redressement des plaques.
 - Tester d'autres variantes de modèles OCR pour améliorer encore la précision de lecture.
- **Post-traitement plus robuste** :
 - Utiliser des règles linguistiques ou des expressions régulières pour valider la structure des matricules reconnus.

Ces pistes devraient permettre d'améliorer les performances du système dans des scénarios réels plus complexes et ainsi favoriser une intégration fiable dans des environnements de stationnement intelligents ou de surveillance urbaine.

3.9 Conclusion

Dans ce chapitre, nous avons détaillé l'implémentation de notre système de détection et de reconnaissance de plaques d'immatriculation, en combinant des modèles avancés comme YOLOv11 pour la détection des plaques et PaddleOCR pour la reconnaissance des caractères. Nous avons abordé les différentes étapes de développement, de la préparation des données à l'évaluation des performances des modèles, en passant par les choix techniques et les ajustements réalisés.

Les résultats obtenus montrent une bonne efficacité du système dans des conditions variées, avec des précisions notables dans la détection des plaques et des places de parking, ainsi qu'une faible erreur dans la reconnaissance OCR. Cependant, des améliorations peuvent être apportées, notamment en enrichissant les données d'entraînement et en affinant les modèles pour mieux gérer des conditions difficiles, telles que des plaques abîmées ou des environnements mal éclairés.

Conclusion générale

Ce mémoire propose un système intelligent de gestion de parking, intégrant la vision par ordinateur et le deep learning. Il se divise en deux modules principaux : la détection des plaques d'immatriculation via YOLOv11, atteignant une précision de 80% (mAP@50), et l'analyse de l'occupation des places de parking, qui affiche une précision de près de 90%. Le modèle YOLOv11, couplé à PaddleOCR, permet également une reconnaissance des caractères avec un taux d'erreur de seulement 5%, démontrant l'efficacité du système dans des conditions réelles variées.

Le système de détection des places de parking a montré une grande robustesse face à des défis comme les variations d'éclairage, les angles de vue différents et les occultations partielles. Cependant, des limitations subsistent, telles que la sensibilité aux conditions extrêmes comme la pluie ou la neige, ainsi qu'aux plaques déformées ou endommagées. Ces défis ont permis de dégager plusieurs pistes d'amélioration, comme l'intégration de données infrarouges pour gérer les faibles conditions de visibilité, ou encore la mise en place de techniques de correction géométrique pour les plaques inclinées.

Les applications pratiques de ce système sont nombreuses. Les gestionnaires de parkings pourraient en tirer profit pour automatiser les processus d'accès, optimisant ainsi l'utilisation de l'espace. Les municipalités bénéficieraient d'une réduction de la congestion et d'une gestion plus efficace des infrastructures. De plus, ce système pourrait offrir aux usagers une expérience de stationnement plus fluide, avec des temps de recherche de places considérablement réduits.

Ce travail ouvre également des perspectives intéressantes pour l'intégration du système dans des environnements de ville intelligente, notamment en couplant la gestion du parking avec la gestion du trafic en temps réel, des applications mobiles pour guider les conducteurs et des plateformes de paiement automatisées. Bien que le système présente encore des axes d'amélioration, il constitue une base solide pour le développement futur de solutions intelligentes dans la gestion des infrastructures urbaines et la mobilité. Ce mémoire propose un système intelligent de gestion de parking, intégrant la vision par ordinateur et le deep learning. Il se divise en deux modules principaux : la détection des plaques d'immatriculation via YOLOv11, atteignant une précision de 87% (mAP@50), et l'analyse de l'occupation des places de parking, qui affiche une précision de près de 97%. Le modèle YOLOv11, couplé à PaddleOCR, permet également une reconnaissance des caractères avec un taux d'erreur de seulement 5%, démontrant l'efficacité du système dans des conditions réelles variées.

Le système de détection des places de parking a montré une grande robustesse face à des défis comme les variations d'éclairage, les angles de vue différents et les occultations partielles. Cependant, des limitations subsistent, telles que la sensibilité aux conditions extrêmes comme la pluie ou la neige, ainsi qu'aux plaques déformées ou endommagées. Ces défis ont permis de dégager plusieurs pistes d'amélioration, comme l'intégration de données infrarouges pour gérer les faibles conditions de visibilité, ou encore la mise en place de techniques de correction géométrique pour les plaques inclinées.

Les applications pratiques de ce système sont nombreuses. Les gestionnaires de parkings pourraient en tirer profit pour automatiser les processus d'accès, optimisant ainsi l'utilisation de l'espace. Les municipalités bénéficieraient d'une réduction de la congestion et d'une gestion plus efficace des infrastructures. De plus, ce système pourrait offrir aux usagers une expérience de stationnement plus fluide, avec des temps de recherche de places considérablement réduits.

Ce travail ouvre également des perspectives intéressantes pour l'intégration du système dans des environnements de ville intelligente, notamment en couplant la gestion du parking avec la gestion du trafic en temps réel, des applications mobiles pour guider les conducteurs et des plateformes de paiement automatisées. Bien que le système présente encore des axes d'amélioration, il constitue une base solide pour le développement futur de solutions intelligentes dans la gestion des infrastructures urbaines et la mobilité.

Bibliographie

- [1] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. Technical report, UC Berkeley, 2014. Tech Report.
- [2] Architecture ssd. <https://developers.arcgis.com/python/latest/guide/how-ssd-works/>. Consulté en 2025.
- [3] Architecture yolo. <https://encord.com/blog/yolo-object-detection-guide/>. Consulté en 2025.
- [4] Image médicale. https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQxPpBwQR_kEByr-3a17_mBXuEnReTHzJefRA&s. Consulté en 2025.
- [5] Reconnaissance faciale. <https://www.cs.cmu.edu/~dst/FaceDemo/resources/tutorial.html>. Consulté en 2025.
- [6] Reconnaissance de plaques avec yolov11. <https://www.ultralytics.com/blog/using-ultralytics-yolo11-for-automatic-number-plate-recognition>. Consulté en 2025.
- [7] Dimensions des plaques d'immatriculation. <https://www.eplaque.fr/plaque-immatriculation/dimension/voiture>. Consulté en 2025.
- [8] Plaque d'immatriculation algérie. https://commons.wikimedia.org/wiki/File:Plaque_d%27immatriculation_-_Alg%C3%A9rie.png. Consulté en 2025.
- [9] CEA.fr. L'intelligence artificielle. 2017. Publié le 21 novembre 2017.
- [10] Dr. V. Devenderan. *Machine Learning, DECAP737*. LPU.
- [11] Jeremy Robert. Deep learning ou apprentissage profond : qu'est-ce que c'est ? *DataScientist*, Septembre 2020.
- [12] Myriam Emilion. Quels sont les différents algorithmes de deep learning ? *Jedha*, Janvier 2025.
- [13] Abdul Vahab, Maruti S. Naik, Prasanna G. Raikar, and S. R. Prasad. Applications of object detection system. *International Research Journal of Engineering and Technology (IRJET)*, 6(4) :4186–4192, 2019.
- [14] Zhengxia Zou, Zhenwei Shi, Yuhong Guo, and Jieping Ye. Object detection in 20 years : A survey. *arXiv preprint arXiv :1905.05055*, 2019.

- [15] J. Donovan E. Neville W. Marroquin S. Tan R. Trafford, R. Linden. Retrofitting rural infrastructure for smart parking and traffic monitoring. In *Proceedings of the 2017 Sensors Applications Symposium (SAS)*, pages 13–15, Glassboro, NJ, USA, March 2017.
- [16] Rahima Khanam* and Muhammad Hussain. Yolov11 : An overview of the key architectural enhancements. *arXiv*, (1), October 24, 2024.
- [17] Rahima Khanam and Muhammad Hussain. Yolov11 : An overview of the key architectural enhancements, 2024.
- [18] Shubhada Deshmukh, Manasi Patwardhan, and Anjali Mahajan. Survey on real-time facial expression recognition techniques. *IET Biometrics*, 5(3) :155–163, 2016.
- [19] Jithmi Shashirangana, Heshan Padmasiri, Dulani Meedeniya, and Charith Perera. Automated license plate recognition : A survey on methods and techniques. *IEEE Access*, 9 :11203–11225, 2020.
- [20] Anastasia Martynova, Mikhail Kuznetsov, Vadim Porvatov, Vladislav Tishin, Andrey Kuznetsov, Natalia Semenova, and Ksenia Kuznetsova. Revising deep learning methods in parking lot occupancy detection. *Artificial Intelligence Research Institute*, 2023. Available at : <https://arxiv.org/abs/2306.04288>.
- [21] D. Wazalwar, E. Oruklu, and J. Saniie. Design flow for robust license plate localization. In *2011 IEEE International Conference on Electro/Information Technology*, pages 1–5. IEEE, 2011.
- [22] Masahiro Daibo. Toroidal vector-potential transformer. In *2017 Eleventh International Conference on Sensing Technology (ICST)*, pages 1–4. IEEE, 2017.
- [23] Yuh-Rau Wang, Wei-Hung Lin, and Shi-Jinn Horng. A sliding window technique for efficient license plate localization based on discrete wavelet transform. *Expert Systems with Applications*, 38(4) :3142–3146, 2011.
- [24] Hong ke Xu, Fu hua Yu, Jia hua Jiao, and Huan sheng Song. A new approach of the vehicle license plate location. In *Sixth International Conference on Parallel and Distributed Computing Applications and Technologies (PDCAT'05)*, pages 1055–1057. IEEE, 2005.
- [25] Rodolfo Zunino and Stefano Rovetta. Vector quantization for license-plate location and image coding. *IEEE Transactions on Industrial Electronics*, 47(1) :159–167, 2000.
- [26] N. Eswar and D. Gowri Shankar Reddy. Morphological operation based vehicle number plate detection. *International Journal of Engineering Research And*, 9(2) :428–433, 2020.
- [27] Zied Selmi, Mohamed Ben Halima, and Adel M. Alimi. Deep learning system for automatic license plate detection and recognition. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, volume 1, pages 1132–1138. IEEE, 2017.

-
- [28] Li Zou, Meng Zhao, Zhengzhong Gao, Maoyong Cao, Huarong Jia, and Mingtao Pei. License plate detection with shallow and deep cnns in complex environments. *Complexity*, 2018 :1–6, 2018.
 - [29] Sergio Montazzolli Silva and Claudio Rosito Jung. License plate detection and recognition in unconstrained scenarios. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 580–596, 2018.
 - [30] Weidong Min, Xiangpeng Li, Qi Wang, Qingpeng Zeng, and Yanqiu Liao. New approach to vehicle license plate location based on new model yolo-l and plate pre-identification. *IET Image Processing*, 13(7) :1041–1049, 2019.
 - [31] Lele Xie, Tasweer Ahmad, Lianwen Jin, Yuliang Liu, and Sheng Zhang. A new cnn-based method for multi-directional car license plate detection. *IEEE Transactions on Intelligent Transportation Systems*, 19(2) :507–517, 2018.
 - [32] Nur-A Alam, Mominul Ahsan, Md Abdul Based, and Julfikar Haider. Intelligent system for vehicles number plate detection and recognition using convolutional neural networks. *Technologies*, 9(1) :9, 2021.
 - [33] M. M. Shaifur Rahman, Mst Shamima Nasrin, Moin Mostakim, and Md Zahangir Alom. Bangla license plate recognition using convolutional neural networks (cnn). *arXiv e-prints*, 2018.
 - [34] Yujie Liu, He Huang, Jinde Cao, and Tingwen Huang. Convolutional neural networks-based intelligent recognition of chinese license plates. *Soft Computing*, 22(7) :2403–2419, 2018.
 - [35] Hui Li, Peng Wang, and Chunhua Shen. Toward end-to-end car license plate detection and recognition with deep neural networks. *IEEE Transactions on Intelligent Transportation Systems*, 20(3) :1126–1136, 2018.
 - [36] Hui Li, Peng Wang, Mingyu You, and Chunhua Shen. Reading car license plates using deep neural networks. *Image and Vision Computing*, 72 :14–23, 2018.
 - [37] Masahiro Daibo. Toroidal vector-potential transformer. In *2017 Eleventh International Conference on Sensing Technology (ICST)*, pages 1–4. IEEE, 2017.
 - [38] Julia Krasevec, Xiaoyi An, Richard Kumapley, France Bégin, and Edward A. Fronillo. Diet quality and risk of stunting among infants and young children in low- and middle-income countries. *Maternal & Child Nutrition*, 13 :e12430, 2017.
 - [39] Saif Ur Rehman, Moiz Ahmad, Asif Nawaz, and Tariq Ali. An efficient approach for vehicle number plate recognition in pakistan. *The Open Artificial Intelligence Journal*, 6(1), 2020.
 - [40] Andrew Mvd. Car plate detection dataset. <https://www.kaggle.com/datasets/andrewmvd/car-plate-detection>, 2020. Consulté le : 26/02/2025.
 - [41] Ultralytics. YOLOv11 : Real-time object detection. <https://github.com/ultralytics/ultralytics>, 2024. Accessed : 2025-02-26.
 - [42] Danushidk507. Paddleocr, 2023.

- [43] Roboflow. Parking spot detector dataset. Accessed : 2024-03-20.
- [44] Python Software Foundation. Aperçu du langage python. <https://www.python.org/doc/essays/blurb/>, 2024. Consulté le 26 mars 2025.
- [45] DataScientest. Google colab : The power of the cloud for machine learning. <https://datascientest.com/en/google-colab-the-power-of-the-cloud-for-machine-learning>, 2024. Consulté le 26 mars 2025.
- [46] Microsoft. Visual studio code - faq. <https://code.visualstudio.com/docs/supporting/FAQ>, 2024. Consulté le 26 mars 2025.
- [47] Ultralytics. Ultralytics yolo docs : Home. <https://docs.ultralytics.com/>, 2023. Consulté le 26 mars 2025.
- [48] Google Brain Team. Tensorflow. <https://www.tensorflow.org/>, 2023. Consulté le 26 mars 2025.
- [49] Meta AI. Pytorch. <https://pytorch.org/>, 2023. Consulté le 26 mars 2025.
- [50] OpenCV Team. Opencv - open computer vision library. <https://opencv.org/>, 2023. Consulté le 26 mars 2025.
- [51] J. D. et al. Hunter. Matplotlib — visualization with python. <https://matplotlib.org/>, 2023. Consulté le 26 mars 2025.
- [52] S. et al. Van der Walt. Numpy. <https://numpy.org/>, 2023. Consulté le 26 mars 2025.
- [53] A. Clark. Pillow (pil fork) 11.1.0 documentation. <https://pillow.readthedocs.io/>, 2023. Consulté le 26 mars 2025.
- [54] PaddlePaddle Team. Paddleocr. <https://github.com/PaddlePaddle/PaddleOCR>, 2023. Consulté le 26 mars 2025.
- [55] Mouad Bensouilah, Mohamed Nadjib Zennir, and Mokhtar Taffar. An alpr system-based deep networks for the detection and recognition. In *ICPRAM*, pages 204–211, 2021.